



UNIVERSIDAD
DE MÁLAGA



TRABAJO FIN DE GRADO

DESARROLLO DE UN SOFTWARE DE AUTOMATIZACIÓN PARA LA ADQUISICIÓN Y PROCESAMIENTO DE DATOS MEDIANTE LIBS

GRADO EN INGENIERÍA ELECTRÓNICA, ROBÓTICA Y MECATRÓNICA

ESCUELA DE INGENIERÍAS INDUSTRIALES

UNIVERSIDAD DE MÁLAGA

Autor: Juan Pedro Gómez Garrido

Tutor: Mónica Trella López

Área de conocimiento: Lenguaje y Sistemas Informáticos

Cotutor: Luisa María Cabalín Robles

Área de conocimiento: Química analítica

Málaga, julio de 2026



UNIVERSIDAD
DE MÁLAGA



DECLARACIÓN DE ORIGINALIDAD DEL PROYECTO/TRABAJO FIN DE GRADO

D./ Dña.:

DNI/Pasaporte: Correo electrónico:

Titulación:

Título del Proyecto/Trabajo:

.....

.....

DECLARA BAJO SU RESPONSABILIDAD

Ser autor/a del texto entregado y que no ha sido presentado con anterioridad, ni total ni parcialmente, para superar materias previamente cursadas en esta u otras titulaciones de la Universidad de Málaga o cualquier otra institución de educación superior u otro tipo de fin.

Así mismo, declara no haber trasgredido ninguna norma universitaria con respecto al plagio ni a las leyes establecidas que protegen la propiedad intelectual, así como que las fuentes utilizadas han sido citadas adecuadamente.

En Málaga, a de de 20.....

Fdo.:

RESUMEN

Este Trabajo Fin de Grado nos presenta el diseño, desarrollo e implementación de un sistema software orientado a la automatización del proceso de adquisición y procesamiento de datos espectrales obtenidos mediante la técnica de espectroscopía de ruptura inducida por láser (LIBS, *Laser-Induced Breakdown Spectroscopy*).

Este trabajo surge con el propósito de proporcionar al grupo de investigación UMALASERLAB una herramienta de software que integre las distintas etapas del proceso experimental: la adquisición del espectro, el almacenamiento de los datos y su posterior procesamiento.

El sistema desarrollado consiste en una aplicación de escritorio implementada en Python, utilizando el framework PyQt6 para la interfaz gráfica de usuario y las bibliotecas NumPy, SciPy y Matplotlib para el procesamiento y la visualización de datos. La aplicación utiliza la API oficial de Avantes para controlar el espectrómetro, pudiendo configurar los parámetros de adquisición, capturar espectros en tiempo real, acumularlos, compararlos y generar mapas de intensidad química en dos dimensiones.

Entre las funcionalidades implementadas se incluyen: normalización y suavizado de señales mediante el filtro de Savitzky-Golay, detección automática de picos espectrales, comparación de espectros con simulaciones de emisión de plasma basadas en datos de la base de datos NIST, mapeado 2D de las intensidades en una longitud de onda seleccionada, y exportación de los resultados en múltiples formatos como TXT, ASCII y Excel.

Los resultados obtenidos de el sistema desarrollado han logrado satisfacer los requisitos fundamentales planteados, mejorando la eficiencia del flujo de trabajo del equipo.

Palabras clave: LIBS, espectroscopía de emisión, automatización, PyQt6, Python, AvaSpec, procesamiento espectral, NIST, perfil de profundidad, mapas 2D.

ABSTRACT

This Final Degree Project presents the design, development, and implementation of a software system aimed at automating the acquisition and processing of spectral data obtained through the technique of Laser-Induced Breakdown Spectroscopy (LIBS).

The project was developed to provide the UMALASERLAB research group with a unified tool capable of integrating the different stages of the experimental process into a single software platform, including spectrum acquisition, data storage, and subsequent processing.

The developed system consists of a desktop application implemented in Python, using the PyQt6 framework for the graphical user interface and the NumPy, SciPy, and Matplotlib libraries for data processing and visualization. The application uses the official Avantes API to control the spectrometer, allowing users to configure acquisition parameters, capture spectra in real time, accumulate and compare spectra, and generate two-dimensional chemical intensity maps.

Among the implemented functionalities are signal normalization and smoothing using the Savitzky–Golay filter, automatic spectral peak detection, comparison of spectra with plasma emission simulations based on data from the NIST database, 2D mapping of intensities at a selected wavelength, and export of results in multiple formats such as TXT, ASCII, and Excel.

The results obtained with the developed system successfully met the fundamental requirements established, improving the efficiency of the research team’s workflow.

Keywords: LIBS, emission spectroscopy, automation, PyQt6, Python, AvaSpec, spectral processing, NIST, 2D maps.

Índice

1. Introducción	1
1.1. Contexto del problema	1
1.2. Motivación del trabajo	2
1.3. Objetivos del TFG	2
1.4. Estructura del documento	3
2. Fundamentos teóricos y antecedentes	5
2.1. Descripción del sistema LIBS	5
2.1.1. Principio de funcionamiento de la técnica LIBS	5
2.1.2. Componentes del sistema LIBS	6
2.2. Procesamiento espectral	7
2.3. Base de datos NIST de espectros atómicos	9
2.4. Espectrómetro AvaSpec-Dual	10
2.5. Software para la adquisición de datos espectrales	11
3. Análisis y diseño del sistema	13
3.1. Requisitos funcionales	13
3.2. Requisitos no funcionales	14
3.3. Tecnologías de desarrollo	14
3.4. API de Avantes	15
3.5. Arquitectura del software	16
4. Implementación	19

4.1. Herramientas y versiones	19
4.2. Estructura del proyecto	20
4.3. Gestión de errores	20
4.4. Módulo de control del espectrómetro	21
4.5. Módulo de procesamiento	22
4.5.1. Filtrado mediante Savitzky-Golay	23
4.6. Módulo de detección automática de picos	24
4.7. Módulo de adquisición	25
4.7.1. Fusión e interpolación de canales espectrales	25
4.8. Módulo de mapeo 2D	26
4.9. Módulos de interfaz gráfica de usuario	27
4.10. Módulo de almacenamiento	27
4.11. Flujo del procesamiento espectral	28
4.12. Visualización de espectros	28
4.13. Pestaña de medida	28
4.14. Pestaña comparativa	28
4.14.1. Simulación NIST	29
4.15. Pestaña de perfil de profundidad	29
4.16. Pestaña de mapas en 2D	30
5. Manual de usuario	31
5.1. Introducción	31
5.2. Instalación y requisitos previos	31
5.3. Conexión del espectrómetro	31
5.4. Configuración de los parámetros de adquisición	32
5.5. Adquisición de espectros	33
5.6. Comparación y análisis de espectros	34
5.7. Perfil de profundidad	35
5.8. Mapeo 2D	35
5.9. Almacenamiento y carga de sesiones	36

6. Resultados experimentales	37
6.1. Ejemplos de espectros obtenidos	38
6.2. Evaluación del sistema	40
7. Conclusiones	43
7.1. Conclusión	43
7.2. Futuras líneas de trabajo	45
Bibliografía	47
Anexos	49
1. Código principal	49

Índice de Tablas

1.	Comparativa entre soluciones software para adquisición LIBS.	11
1.	Versiones de las herramientas y bibliotecas empleadas.	19

Índice de Ilustraciones

1.	Esquema de la instrumentación LIBS.	7
1.	Diagrama UML. Arquitectura del software.	17
1.	Diagrama de secuencia de llamadas del módulo de control del espectrómetro. .	22
1.	Interfaz gráfica del sistema.	32
2.	Interfaz gráfica de la pestaña de comparación.	35
1.	Ejemplo de espectro LIBS adquirido con el sistema desarrollado, con aplicación del filtro de Savitzky-Golay.	38
2.	Vista de la pestaña comparativa con varios espectros superpuestos.	39
3.	Perfil de profundidad para la longitud de onda de una línea espectral seleccionada.	39
4.	Mapa de intensidad química en 2D generado para la longitud de onda de una línea espectral seleccionada.	40
5.	Espectros simulados con datos del NIST.	40

Capítulo 1

Introducción

1.1. Contexto del problema

La espectroscopia de ruptura inducida por láser, conocida también como LIBS (*Laser-Induced Breakdown Spectroscopy*), es una técnica para realizar análisis de la composición química de un material[1]. Se realiza a través de la obtención de su espectro generado mediante un láser. Cuando este láser incide sobre la superficie del material, se produce una ablación del material, formándose un plasma compuesto por átomos, iones y electrones excitados. Durante la relajación se emite radiación electromagnética en diferentes longitudes de onda[2].

La radiación emitida por el plasma puede ser recogida mediante un sistema óptico y analizada con un espectrómetro, que descompone la luz en sus diferentes componentes espectrales, dando como resultado un espectro de emisión que contiene información sobre la composición química del material[3]. Gracias a estas características, la técnica LIBS se utiliza en múltiples aplicaciones como son análisis de materiales industriales[4], control de calidad en fabricación, caracterización de suelos y minerales, estudios medioambientales y arqueológicos[5], así como en aplicaciones espaciales de análisis remoto[2].

En sistemas experimentales basados en LIBS es habitual trabajar con grandes cantidades de datos espectrales, especialmente cuando se realizan medidas repetidas o análisis espaciales sobre diferentes puntos de una muestra[1]. Por esta razón, resulta fundamental disponer de sistemas eficientes de adquisición y procesamiento de datos que permitan registrar los espectros de manera fiable, almacenarlos correctamente y facilitar su análisis posterior.

1.2. Motivación del trabajo

En el grupo de investigación UMALASERLAB de la Universidad de Málaga se emplean diversos espectrómetros de la familia AvaSpec de Avantes [6], sistemas láser pulsados y etapas motorizadas de posicionamiento para la caracterización elemental de muestras. Actualmente, la coordinación entre estos dispositivos se realiza de forma manual, lo que limita la ejecución del proceso experimental y la eficiencia en la adquisición de datos.

La adquisición de espectros, su almacenamiento y el análisis posterior se realizan frecuentemente usando diferentes aplicaciones, lo que dificulta el trabajo. Cuando se requieren mediciones prolongadas o experimentos que implican un gran número de espectros, la falta de automatización constituye un obstáculo significativo[4].

La motivación principal de este trabajo radica en mejorar la eficiencia del proceso experimental mediante el desarrollo de un sistema software que permita automatizar la adquisición, el almacenamiento y el procesamiento de los datos. De esta forma, se busca facilitar el trabajo experimental y ofrecer una herramienta flexible que pueda ser ampliada en el futuro para integrar el control coordinado de todos los dispositivos hardware del sistema.

1.3. Objetivos del TFG

El objetivo principal de este Trabajo Fin de Grado es desarrollar un sistema software que permita automatizar el proceso de adquisición, almacenamiento y procesamiento inicial de datos espectrales obtenidos mediante un sistema de espectroscopía LIBS.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

- Analizar el funcionamiento de los sistemas de espectroscopía LIBS y los dispositivos de adquisición de datos espectrales empleados en el laboratorio UMALASERLAB[2].
- Estudiar la interfaz de programación (API) del espectrómetro AvaSpec-Dual de Avantes para determinar el mecanismo de integración en la aplicación[7].
- Diseñar una arquitectura software modular que permita la integración de las diferentes funcionalidades necesarias para el control del sistema de adquisición.

- Implementar un sistema que permita configurar los parámetros de adquisición, capturar espectros en tiempo real y almacenar los datos de forma estructurada.
- Desarrollar módulos de procesamiento espectral que incluyan normalización, filtrado[8], detección de picos y comparación con bases de datos de referencia (NIST)[9].
- Implementar herramientas de visualización de espectros individuales, comparación entre espectros y generación de mapas de intensidad química en dos dimensiones.
- Evaluar el funcionamiento del sistema desarrollado mediante pruebas que permitan comprobar su utilidad en el entorno de laboratorio.

1.4. Estructura del documento

El presente documento se organiza en los siguientes capítulos:

Capítulo 1. Introducción.

Se explica como surge este trabajo y cuales son sus objetivos.

Capítulo 2. Fundamentos teóricos y antecedentes.

Se describe el sistema LIBS del laboratorio, el espectrómetro AvaSpec-Dual y la arquitectura del sistema completo.

Capítulo 3. Análisis y diseño del sistema.

Se describe detalladamente el sistema, incluyendo las tecnologías empleadas y los aspectos más relevantes del código desarrollado.

Capítulo 4. Implementación.

Se presenta todos los módulos recogen las conclusiones del trabajo y se plantean líneas de trabajo futuro.

Capítulo 5. Conclusiones.

Se recogen las conclusiones del trabajo y se plantean líneas de trabajo futuro.

Capítulo 6. Conclusiones.

Se recogen las conclusiones del trabajo y se plantean líneas de trabajo futuro.

Capítulo 7. Conclusiones.

Se recogen las conclusiones del trabajo y se plantean líneas de trabajo futuro.

Capítulo 2

Fundamentos teóricos y antecedentes

2.1. Descripción del sistema LIBS

2.1.1. Principio de funcionamiento de la técnica LIBS

El principio de funcionamiento de la técnica LIBS consiste en que un pulso láser incide sobre la superficie de la muestra[1]. Si la densidad de energía del pulso es suficientemente alta, es cuando se produce la ruptura de enlaces químicos y la vaporización de una pequeña cantidad de material. Este material vaporizado es el que da lugar a la formación de un plasma, también llamado pluma de plasma, que se expande rápidamente en el entorno cercano a la superficie del material[2].

Durante la expansión y enfriamiento del plasma, los átomos e iones excitados regresan a estados energéticos más estables, emitiendo radiación electromagnética en longitudes de onda características de cada elemento químico. Esta radiación constituye la huella espectral que permite la identificación cualitativa y, en ciertos casos, cuantitativa de los elementos presentes[3].

Cabe destacar que la emisión del plasma no es estacionaria en el tiempo. Durante los primeros nanosegundos tras el pulso predomina una emisión continua de fondo de alta intensidad que enmascara las líneas atómicas características. A medida que el plasma se enfría, esta emisión decae, haciendo distinguibles las líneas espectrales. Por esta razón es necesario añadir un retardo configurable entre el disparo y el inicio de adquisición, de manera que el

espectrómetro capture la señal en la ventana temporal óptima[1].

La señal emitida por el plasma es recogida usando un sistema óptico, generalmente compuesto por lentes o fibras ópticas, y dirigida hacia un espectrómetro. Los datos espectrales son procesados posteriormente usando un software especializado[1].

2.1.2. Componentes del sistema LIBS

Un sistema LIBS típico está compuesto por varios elementos fundamentales[2]:

- **Láser pulsado:** opera habitualmente a una longitud de onda de 1064 nm (Nd:YAG), con energías por pulso del orden de decenas de milijulios y duraciones del orden de los nanosegundos.
- **Sistema óptico de focalización:** lentes o espejos que concentran el haz sobre la muestra para alcanzar la densidad de energía necesaria.
- **Sistema de recolección de emisión:** fibras ópticas o lentes de colimación que capturan la luz del plasma y la conducen al espectrómetro.
- **Espectrómetro:** dispositivo que descompone la luz en sus componentes espectrales y las registra mediante un detector.
- **Sistema de adquisición y control:** ordenador con software específico para la captura, procesamiento y análisis de los espectros.

En la figura 1 se puede observar un esquema del sistema LIBS.

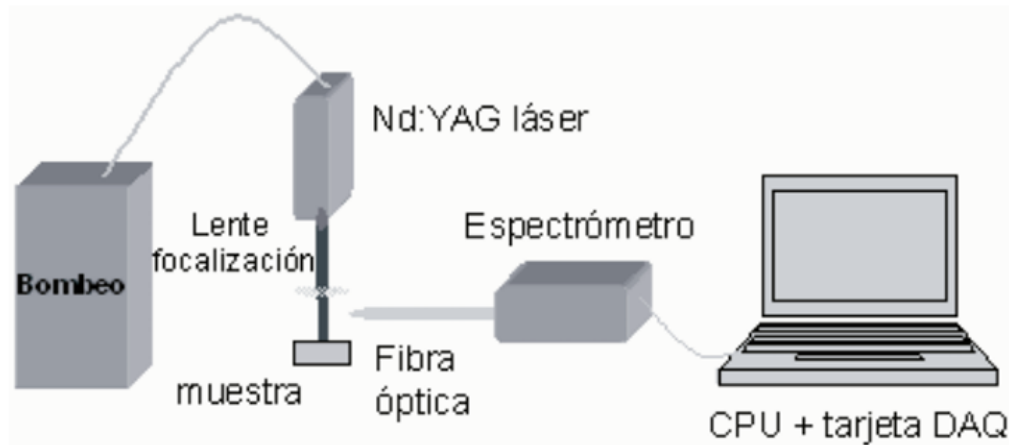


Figura 1. Esquema de la instrumentación LIBS.

Su flujo de procesamiento se puede definir con las siguientes etapas:

- Generación del pulso láser y focalización sobre la muestra.
- Emisión del plasma y captura de la radiación mediante fibra óptica.
- Descomposición espectral y detección.
- Transmisión de datos al ordenador.
- Adquisición, procesamiento, visualización y almacenamiento mediante el software desarrollado.

La integración del software en este flujo permite automatizar las últimas dos etapas, eliminando la intervención manual en el proceso de adquisición y liberando al operador para tareas de análisis e interpretación[4].

2.2. Procesamiento espectral

El procesamiento de señales espectrales comprende el conjunto de técnicas matemáticas y computacionales aplicadas sobre los datos crudos obtenidos por el espectrómetro con el objetivo de mejorar su calidad, extraer información relevante y facilitar la interpretación de los resultados. En el contexto de la espectroscopía LIBS, esta etapa es especialmente crítica debido a las características propias de la señal: alta densidad de líneas espectrales en rangos

de longitud de onda reducidos, presencia de ruido instrumental y fluctuaciones entre disparos causadas por la naturaleza estocástica del plasma[1].

La señal adquirida por el espectrómetro contiene, además de la información espectral de interés, una serie de contribuciones no deseadas que deben ser corregidas antes del análisis. Entre ellas destaca la línea de base instrumental, un desplazamiento sistemático de la señal que puede deberse a la corriente de oscuridad del detector o a la emisión continua residual del plasma. Su corrección es el primer paso habitual del flujo de procesamiento y permite que las intensidades medidas sean comparables entre sesiones realizadas en distintas condiciones experimentales[2].

Las tres técnicas de procesamiento espectral más extendidas en el procesamiento espectral son las siguientes:

- **Suavizado y filtrado:** reduce el ruido de alta frecuencia superpuesto a la señal espectral. Entre los métodos más empleados se encuentran la media móvil, el filtro gaussiano y el filtro de Savitzky-Golay.[8] Este último es el preferido en espectroscopía porque, al realizar un ajuste polinómico local por mínimos cuadrados, preserva la forma, anchura e intensidad de los picos espectrales mejor que los filtros de promediado simple, que tienden a distorsionarlos.
- **Corrección de línea de base:** elimina el fondo continuo de la señal para que las intensidades de las líneas espectrales sean atribuibles únicamente a la emisión atómica del plasma. Los métodos más habituales son la sustracción del mínimo global, la estimación polinómica del fondo y los algoritmos iterativos como SNIP (*Statistics-sensitive Non-linear Iterative Peak-clipping*). Una corrección adecuada de la línea de base es imprescindible para obtener resultados cuantitativos fiables[1].
- **Normalización:** escala la señal a un rango de referencia común para eliminar la variabilidad entre disparos causada por fluctuaciones en la energía del láser o en la eficiencia de acoplamiento óptico. Las estrategias más empleadas son la normalización por el valor máximo del espectro, por la intensidad de una línea de referencia interna de concentración conocida, y por la integral total del espectro. Esta última es especialmente robusta frente a cambios en las condiciones del plasma[2].

2.3. Base de datos NIST de espectros atómicos

La base de datos de espectros atómicos del Instituto Nacional de Estándares y Tecnología NIST (*National Institute of Standards and Technology*) es la referencia internacional más completa y ampliamente utilizada para la identificación elemental en espectroscopía de emisión[9]. Mantenida y actualizada de forma continua por el NIST Physical Measurement Laboratory, recoge los datos espectroscópicos de prácticamente todos los elementos de la tabla periódica, incluyendo longitudes de onda de emisión, niveles de energía, probabilidades de transición y coeficientes de Einstein para decenas de miles de líneas espectrales atómicas e iónicas.

Los datos que proporciona la base de datos NIST relevantes para el análisis LIBS son principalmente los siguientes:

- **Longitud de onda de la transición** (λ): posición espectral de cada línea de emisión en el vacío o en el aire, expresada en nanómetros o angstroms. Es el dato fundamental para identificar un elemento comparando la posición de sus líneas con las del espectro experimental.
- **Intensidad relativa**: valor proporcional a la probabilidad de que una transición concreta sea observable bajo condiciones de plasma típicas. Permite estimar qué líneas de un elemento serán más intensas y por tanto más fácilmente detectables en el espectro experimental.
- **Niveles de energía** (E_k, E_i): energías del nivel superior e inferior de la transición, expresadas en cm^{-1} . Son necesarias para calcular la distribución de Boltzmann de la población de estados excitados y, por tanto, para simular el espectro de emisión a una temperatura electrónica dada.
- **Probabilidad de transición** (A_{ki}): coeficiente de Einstein de emisión espontánea, que cuantifica la probabilidad por unidad de tiempo de que un átomo en el nivel superior decaiga al inferior emitiendo un fotón. Junto con los niveles de energía, permite calcular la intensidad absoluta de cada línea bajo el modelo de equilibrio termodinámico local (LTE).

- **Grado de ionización:** indica si la línea corresponde a un átomo neutro (I), una vez ionizado (II) o en estados de ionización superiores. En un plasma LIBS, la distribución entre estados de ionización depende de la temperatura electrónica, por lo que esta información es fundamental para interpretar correctamente el espectro.

En el contexto de este trabajo, la base de datos NIST se utiliza a través de la biblioteca Python **SimulatedLIBS**[9], que proporciona una interfaz de consulta tanto en modo estático, mediante una caché local de los datos más frecuentes, como en modo dinámico, realizando consultas en tiempo real al servidor web de NIST.

La comparación visual entre el espectro experimental y el simulado a partir de los datos NIST permite al investigador identificar los elementos presentes en la muestra, verificar la asignación de las líneas espectrales observadas y estimar de forma cualitativa las condiciones del plasma en el momento de la adquisición.

2.4. Espectrómetro AvaSpec-Dual

En el laboratorio UMALASERLAB se usan varios espectómetros de la familia Avantes, entre ellos el AvaSpec-Dual. Este instrumento cuenta con dos canales espectrales independientes, lo que permite cubrir un rango espectral ampliado. Cada canal cuenta con su propio detector CCD lineal, haciendo posible la adquisición simultánea en dos rangos de longitud de onda complementarios, incrementando la cobertura total del sistema sin tener que reconfigurarlo entre medidas[6].

Entre sus principales características se encuentran:

- Conectividad mediante interfaz USB de alta velocidad.
- Configuración del tiempo de integración por software.
- Soporte para disparo externo por hardware para sincronización con el láser.
- Capacidad de adquirir espectros en modo continuo o a petición del software.

La comunicación entre el espectrómetro y el ordenador se realiza a través de la DLL de Avantes[7].

2.5. Software para la adquisición de datos espectrales

Actualmente existen soluciones software destinadas a la adquisición de datos espectrales. El propio fabricante Avantes distribuye su software AvaSoft, que permite controlar sus espectrómetros y ver espectros en tiempo real. Sin embargo, estas herramientas están hechas para un uso básico y no ofrecen funciones específicas. Por otro lado, entornos como LabVIEW permiten desarrollar sistemas de adquisición, a costa de una alta curva de aprendizaje y licencias costosas. En consecuencia, los grupos de investigación deben acabar por recurrir a combinar varias herramientas, fragmentando su flujo de trabajo y dificultando la reproducibilidad de los experimentos.

La tabla 1 recoge una comparativa entre las soluciones existentes y el sistema desarrollado en este trabajo.

Funcionalidad	AvaSoft	LabVIEW	Software desarrollado
Adquisición en tiempo real	Sí	Sí	Sí
Control del espectrómetro Avantes	Sí	Parcial	Sí
Acumulación de disparos	No	Manual	Sí (3 modos)
Filtrado Savitzky-Golay	No	Manual	Sí
Comparación de espectros	No	Manual	Sí
Simulación NIST integrada	No	No	Sí
Perfil de profundidad	No	Manual	Sí
Mapeo 2D de intensidad	No	Manual	Sí
Exportación a ficheros	Sí	Sí	Sí
Carga de sesiones anteriores	No	Manual	Sí
Código abierto / sin licencia	No	No	Sí

Tabla 1. Comparativa entre soluciones software para adquisición LIBS.

De la comparativa se desprende que ninguna de las soluciones existentes cubre de forma integrada el flujo de trabajo completo del laboratorio UMALASERLAB. AvaSoft cubre únicamente la adquisición básica, y LabVIEW requiere un desarrollo a medida para cada funcionalidad adicional. El sistema desarrollado en este trabajo unifica todas las etapas en una

única herramienta de código abierto, específicamente adaptada a las necesidades del grupo de investigación.

La automatización de este proceso nos permite mejorar la ejecución de los experimentos, permitiendo reducir la intervención del operador y aumentar la eficiencia en la obtención. A su vez, nos facilita integrar el sistema de medida con herramientas de procesamiento y visualización de datos.

Capítulo 3

Análisis y diseño del sistema

3.1. Requisitos funcionales

Los requisitos funcionales del sistema definen qué debe hacer el sistema. Describen funciones, comportamientos o servicios específicos que el sistema debe proporcionar al usuario:

- **Conexión con el espectrómetro:** el sistema debe ser capaz de detectar, conectar y desconectar los espectrómetros AvaSpec disponibles[6].
- **Configuración de parámetros:** el usuario debe poder configurar el tiempo de integración, el número de disparos y el modo de disparo.
- **Adquisición en tiempo real:** el sistema debe capturar espectros en tiempo real y mostrarlos en la interfaz gráfica de forma continuada.
- **Modos de acumulación:** el sistema debe soportar tres modos de visualización: espectro individual, acumulación por suma y acumulación por promedio.
- **Almacenamiento de espectros:** el sistema debe permitir guardar los espectros capturados en los formatos TXT, ASCII y Excel.
- **Carga de sesiones:** el sistema debe permitir cargar medidas de sesiones anteriores para compararlos.

- **Comparación de espectros:** el usuario debe poder visualizar y comparar múltiples espectros simultáneamente.
- **Identificación elemental:** el sistema debe poder superponer líneas de emisión de elementos de la tabla periódica obtenidas de la base de datos NIST[9].
- **Mapas 2D:** el sistema debe poder generar mapas de intensidad química en dos dimensiones para una longitud de onda seleccionada.[4]
- **Normalización y filtrado:** el sistema debe aplicar normalización y filtrado Savitzky-Golay[8] a los espectros adquiridos.

3.2. Requisitos no funcionales

Los requisitos no funcionales describen cómo debe comportarse el sistema. Describen atributos de calidad, restricciones o características globales:

- **Rendimiento:** la interfaz gráfica debe actualizar los espectros en tiempo real sin bloquear el hilo principal de la aplicación[10].
- **Modularidad:** el código debe organizarse en módulos independientes y reutilizables.
- **Portabilidad:** el sistema debe funcionar en plataformas Windows de 64 bits, que es el sistema operativo habitual en los laboratorios de la universidad.
- **Extensibilidad:** la arquitectura debe facilitar la incorporación futura de módulos adicionales, como el control de etapas motorizadas.

3.3. Tecnologías de desarrollo

El desarrollo se ha realizado mediante el lenguaje de programación Python, debido a su simplicidad, versatilidad y amplia disponibilidad de bibliotecas. Mientras que para la construcción de la interfaz gráfica he usado PyQt6, un framework basado en Qt.

En el ámbito de cálculo numérico, se han usado las bibliotecas NumPy y SciPy. NumPy nos ofrece estructuras eficientes para el manejo de matrices, mientras que SciPy amplía estas capacidades incorporando algoritmos científicos y funciones avanzadas de análisis.

Por último, para la representación gráfica de resultados y visualización de datos se ha utilizado Matplotlib, una biblioteca que genera gráficas de alta calidad y facilita la visualización de los resultados de ejecución de la aplicación.

3.4. API de Avantes

La DLL de Avantes[7] es una biblioteca de enlace dinámico que proporciona una interfaz C a la que se puede acceder desde Python mediante el módulo `ctypes`. Esta elección resulta bastante adecuada en entornos de laboratorio, donde la instalación de cadenas de compilación pueden resultar complejas o no estar disponibles. Las principales funciones de la API usadas en este trabajo son:

- `AVS_Init`: inicializa la comunicación con los espectrómetros conectados.
- `AVS_Activate`: activa y devuelve un *handle* al espectrómetro seleccionado.
- `AVS_GetNumPixels`: obtiene el número de píxeles del detector.
- `AVS_GetLambda`: obtiene el vector de longitudes de onda.
- `AVS_PrepareMeasure`: configura los parámetros de medida mediante la estructura `MeasConfigType` como son tiempo de integración, número de promedios, modo de disparo.
- `AVS_Measure`: realiza una adquisición.
- `AVS_PollScan`: consulta si hay un nuevo espectro disponible.
- `AVS_GetScopeData`: recupera los datos del espectro adquirido.
- `AVS_Deactivate`: desactiva el espectrómetro.

El uso de `AVS_PollScan` en un bucle de espera activa permite sincronizar la lectura con el disparo externo sin bloquear el hilo del sistema de forma indefinida. Para proteger la aplicación ante la ausencia del disparo, se establece un tiempo máximo de espera, tras el cual la función de lectura devuelve `None`, evitando que el hilo de adquisición se quede bloqueado [7].

3.5. Arquitectura del software

La aplicación sigue una arquitectura modular adaptado al entorno PyQt6 [10] basada en el patrón de diseño Modelo-Vista-Controlador (MVC). Este enfoque separa los datos y la lógica de negocio (modelo) de su representación visual (vista) y del módulo que gestiona los eventos y la interacción con el usuario (controlador). Los componentes principales del sistema desarrollado son:

- **Capa de hardware** (`Spectrometer`): encapsula toda la comunicación con la API de Avantes[6].
- **Capa de procesamiento** (`Processor`): implementa los algoritmos de tratamiento de señal, incluyendo el filtro Savitzky-Golay[8].
- **Hilos de trabajo** (`TriggerThread`, `MapThread`): ejecutan las operaciones de adquisición emitiendo señales Qt (`pyqtSignal`) en segundo plano para no bloquear la interfaz gráfica[10].
- **Capa de presentación** (`PlotTab`, `MeasureTab`, `CompareTab`): componentes de la interfaz gráfica.
- **Controlador principal** (`App`): coordina la interacción entre los distintos módulos y gestiona el estado global de la aplicación.

La elección del patrón MVC responde a la necesidad de que distintos componentes puedan actuar de forma independiente. Por ejemplo, sustituir el espectrómetro requeriría solo modificar la capa de hardware, sin afectar a la capa de procesamiento ni a la interfaz gráfica.

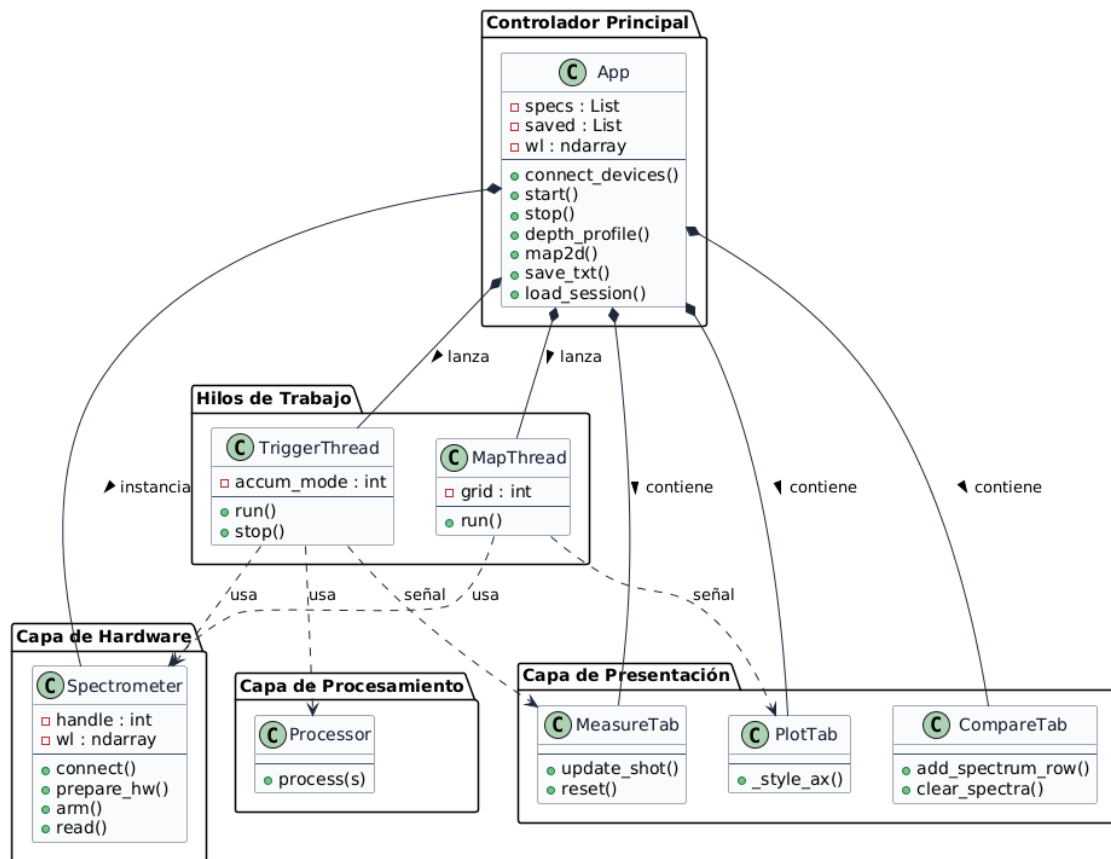


Figura 1. Diagrama UML. Arquitectura del software.

Capítulo 4

Implementación

4.1. Herramientas y versiones

Las versiones exactas de las herramientas y bibliotecas empleadas se recogen en la tabla 1, con el objetivo de garantizar la reproducibilidad del entorno de desarrollo y facilitar futuras actualizaciones del sistema.

Herramienta / Biblioteca	Versión	Propósito
Python	3.11.9	Lenguaje de implementación
PyQt6	6.7.0	Interfaz gráfica de usuario
Matplotlib	3.9.0	Visualización de espectros
NumPy	1.26.4	Procesamiento numérico
SciPy	1.13.0	Filtrado y detección de picos
openpyxl	3.1.2	Exportación a formato Excel
SimulatedLIBS	1.1.0	Simulación de espectros NIST
ctypes	(stdlib)	Acceso a la DLL de Avantes
Visual Studio Code	1.89	Editor de código

Tabla 1. Versiones de las herramientas y bibliotecas empleadas.

4.2. Estructura del proyecto

El código fuente del proyecto se organiza en un único fichero principal `main.py`, que contiene la definición de todas las clases y el punto de entrada de la aplicación. Esta decisión de mantener el código en un único módulo es debido que al tratarse de una aplicación de escritorio con un número de clases acotado y dependencias estrechas entre ellas, la separación en múltiples ficheros habría traído complejidad en la importación sin aportar beneficios para la organización. No obstante, la estructura interna del fichero sigue un orden lógico que refleja las capas de la arquitectura MVC: primero las constantes y configuración global, después las clases de hardware, procesamiento, hilos de trabajo, presentación y finalmente el controlador principal `App`.

Adicionalmente, el módulo `avaspec.py` encapsula la carga de la DLL de Avantes y la definición de las estructuras C necesarias para la comunicación con el espectrómetro, manteniendo este código de bajo nivel separado de la lógica principal de la aplicación.

4.3. Gestión de errores

El sistema implementa varios mecanismos de gestión de errores para garantizar su robustez en condiciones reales de laboratorio:

- **Timeout de adquisición:** la función `read` de la clase `Spectrometer` devuelve `None` si el espectrómetro no responde antes de que transcurra un tiempo de espera máximo, para evitar que la adquisición se quede bloqueada.
- **Validación de dispositivos:** la función `connect_devices` verifica que `AVS_GetNrOfDevices` devuelve al menos un dispositivo antes de intentar la activación, mostrando un mensaje de error en caso contrario.
- **Protección del autoguardado:** antes de escribir el fichero TXT automático al finalizar una sesión, el sistema comprueba que no existe ya un fichero con el mismo nombre, evitando sobreescrituras accidentales de sesiones anteriores.
- **Compatibilidad de ejes:** al cargar sesiones anteriores con un vector de longitudes de

onda de distinta longitud al actual, el sistema interpola automáticamente los espectros cargados al eje del espectrómetro activo, permitiendo comparar sesiones adquiridas con distintas configuraciones.

4.4. Módulo de control del espectrómetro

En la clase **Spectrometer** se encapsula toda la comunicación con el espectrómetro AvaSpec usando la DLL de Avantes, a la cual se accede mediante **ctypes**[7]. A continuación se enumeran las funciones más relevantes:

- En la función **connect** se activa el dispositivo, obteniendo el número de píxeles detectados, con el que sacamos el tamaño del vector de longitudes de onda. Este vector será clave para representr correctamente el eje horizontal de los espectros[7].
- En la función **prepare hw** se realiza la configuración de los parámetros del espectrómetro. La usando la estructura **MeasConfigType** de la API[7].
- Por último **arm** se usa para inicializar la medida, mientras que **read** ejecuta la lectura usando un tiempo de espera máximo configurable.

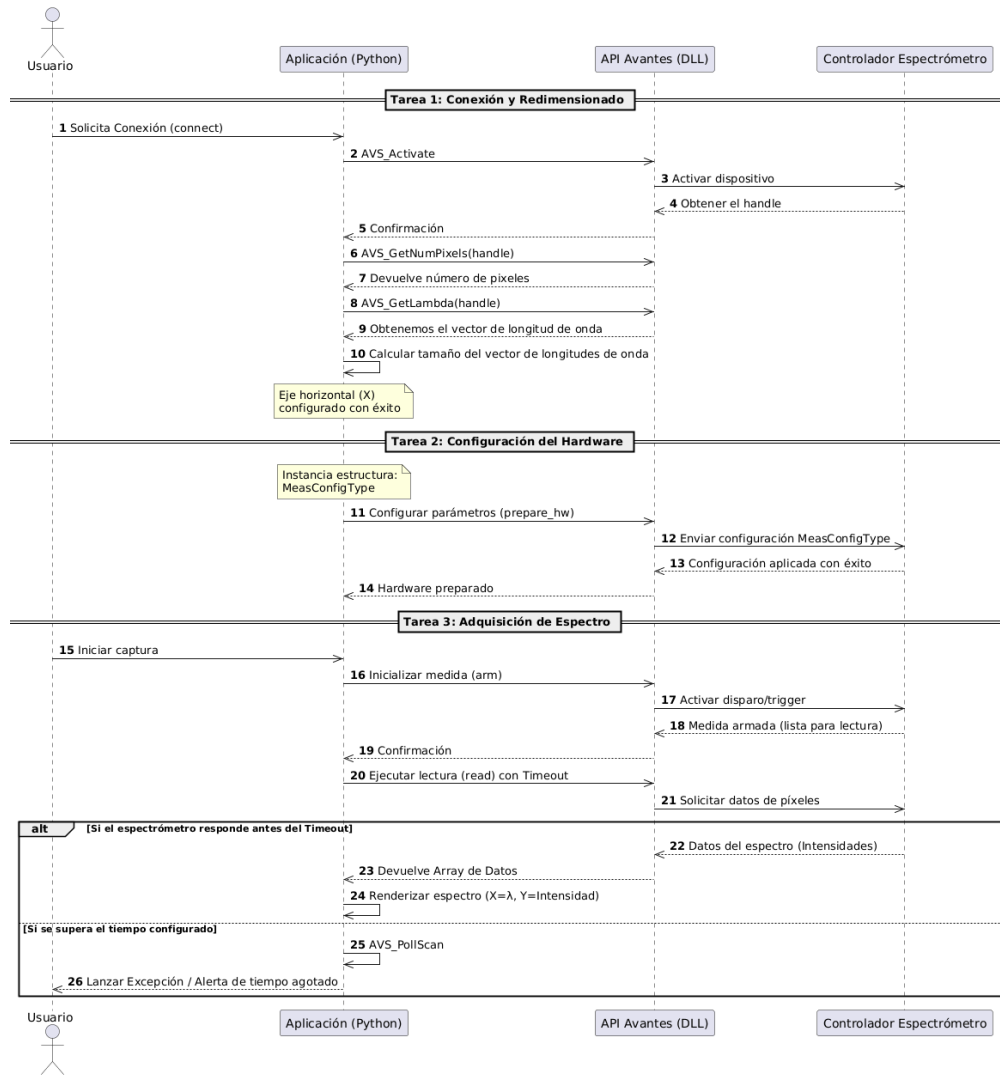


Figura 1. Diagrama de secuencia de llamadas del módulo de control del espectrómetro.

4.5. Módulo de procesamiento

La clase `Processor` implementa un procesamiento básico en los espectros[11]. Aplicando varias operaciones como la sustracción del mínimo para corregir la línea base, junto con una división para normalizar y el filtro de Savitzky-Golay[8]:

El comando `savgol_filter(s, 11, 3)` aplica el filtro usando una ventana de 11 puntos y un polinomio de grado 3, parámetros que han demostrado buen rendimiento para señales espectrales LIBS de resolución moderada[8].

4.5.1. Filtrado mediante Savitzky-Golay

Las señales obtenidas mediante LIBS suelen tener un poco de ruido asociado, fluctuaciones de plasma y variaciones instrumentales[1]. Por esto, es necesario aplicar técnicas de suavizado que nos permitan reducirlo.

Para este objetivo se ha implementado un filtro basado en el algoritmo de Savitzky-Golay[8], usado debido a su capacidad de preservar la forma, anchura e intensidad de los picos espectrales. Debido a que realiza el ajuste polinómico local mediante mínimos cuadrados sobre una ventana deslizante de puntos de la señal.

El modelo matemático aplicado para este ajuste es el siguiente:

$$y(x) = \sum_{k=0}^p a_k x^k$$

donde p representa el grado del polinomio utilizado y a_k son los coeficientes obtenidos mediante el ajuste por mínimos cuadrados dentro de cada ventana de cálculo.

En la implementación desarrollada se utiliza la función `savgol_filter` incluida en la biblioteca SciPy[11].

En nuestro caso usamos una ventana de 11 muestras y polinomio de grado 3. Estos parámetros se seleccionaron experimentalmente debido a que proporcionan un compromiso adecuado entre reducción de ruido y conservación de la resolución espectral de las señales LIBS adquiridas en el laboratorio[8].

El empleo de ventanas de tamaño superior genera un suavizado excesivo que puede reducir la intensidad de líneas espectrales estrechas, mientras que el uso de polinomios de orden demasiado elevado puede introducir oscilaciones artificiales en la señal procesada[8].

La aplicación de este filtro mejora considerablemente la estabilidad visual de los espectros mostrados en tiempo real y facilita procesos posteriores como la detección automática de picos espectrales, la comparación entre medidas y la identificación elemental mediante la base de datos NIST[9].

4.6. Módulo de detección automática de picos

Uno de los objetivos específicos del sistema es la detección automática de picos espectrales sobre los espectros adquiridos. Esta funcionalidad resulta especialmente útil en la pestaña comparativa, donde permite identificar visualmente las líneas de emisión más relevantes sin necesidad de inspeccionar manualmente el espectro punto a punto.

La detección se implementa mediante la función `find_peaks` incluida en la biblioteca SciPy[11], que localiza los máximos locales de la señal aplicando dos criterios de filtrado simultáneos:

- **Prominencia mínima:** se considera pico únicamente aquél cuya altura sobre el entorno local supera un umbral calculado dinámicamente como el 8 % del rango total de la señal:

$$\text{umbral} = 0,08 \cdot (\max(s) - \min(s))$$

Este enfoque relativo hace que el detector sea robusto frente a variaciones de escala entre sesiones con distintos tiempos de integración o materiales con diferentes eficiencias de emisión.

- **Distancia mínima entre picos:** se impone una separación mínima de 10 píxeles entre picos detectados consecutivos, evitando la detección múltiple sobre un mismo perfil de línea espectral ensanchado por efectos instrumentales.

De todos los picos detectados, el sistema selecciona los 6 de mayor prominencia y los representa en la gráfica comparativa mediante marcadores triangulares, acompañados de una anotación con la longitud de onda correspondiente en nanómetros. Los picos se calculan una única vez por espectro y se almacenan en una caché interna (`__peak_cache`) indexada por posición en la lista de espectros, evitando recálculos innecesarios cada vez que se actualiza la visualización.

La figura 2 ilustra el resultado de esta detección sobre un conjunto de espectros superpuestos, donde los marcadores permiten identificar de forma inmediata las líneas de emisión dominantes y compararlas con las posiciones tabuladas en la base de datos NIST[9].

4.7. Módulo de adquisición

La clase `TriggerThread` se implementa como un hilo independiente[10] para evitar que bloquee la interfaz gráfica. Emitiendo señales Qt cuando hay un nuevo espectro disponible.

En caso de que haya más de un canal, estos se interpolan[12]. En el caso del AvaSpec-Dual que cuenta con dos canales, se interpolan en un eje común de longitud de onda de 2000 puntos y se promedia, obteniendo un espectro unificado contando con mayor cobertura.

El hilo de adquisición también cuenta con tres modos:

- **Modo sin acumulación:** Este es el más común, no aplica ninguna modificación al espectro
- **Modo suma acumulada):** Aquí los espectros se van sumando con cada disparo.
- **Modo promedio acumulado):** En este modo se aplica una media que se actualiza con cada disparo usando la siguiente fórmula recursiva: $\bar{s}_n = \bar{s}_{n-1} \cdot \frac{n-1}{n} + \frac{s_n}{n}$.

La fórmula recursiva del modo promedio resulta más eficiente que recalcular la media aritmética completa en cada disparo, ya que evita la necesidad de almacenar los espectros previos en memoria, reduciendo el coste operacional.

4.7.1. Fusión e interpolación de canales espectrales

Cuando el sistema opera con un espectrómetro de múltiples canales, cada canal devuelve un vector de longitudes de onda y un vector de intensidades independientes. Debido a que cada canal tiene un rango distinto, normalmente solo se solapan parcialmente, debiendo unificarse en un único espectro antes de visualizarlo y procesarlo.

El procedimiento de fusión implementado en `TriggerThread` sigue los siguientes pasos[12]:

1. Se determinan los extremos del rango espectral combinado:

$$\lambda_{\min} = \min_i \{\lambda_i[0]\}, \quad \lambda_{\max} = \max_i \{\lambda_i[-1]\}$$

donde λ_i es el vector de longitudes de onda del canal i .

2. Se construye un eje común de longitud de onda uniforme con 2000 puntos equiespaciados en el rango $[\lambda_{\min}, \lambda_{\max}]$ mediante:

$$\lambda_c = \text{np.linspace}(\lambda_{\min}, \lambda_{\max}, 2000)$$

3. Cada espectro individual s_i se interpola linealmente sobre el eje común λ_c usando `np.interp`, que asigna a cada punto de λ_c la intensidad interpolada a partir de los puntos conocidos del canal correspondiente:

$$\hat{s}_i(\lambda_c) = \text{np.interp}(\lambda_c, \lambda_i, s_i)$$

4. El espectro unificado final se obtiene como la media aritmética de los espectros interpolados de todos los canales:

$$s_{\text{out}} = \frac{1}{N} \sum_{i=1}^N \hat{s}_i(\lambda_c)$$

donde N es el número de canales activos.

Este procedimiento garantiza que el espectro resultante cubra el rango espectral completo de todos los canales, contando con una resolución de 2000 puntos. En las regiones de solapamiento entre canales, el promediado además reduce el ruido respecto a cualquiera de los canales individuales.

4.8. Módulo de mapeo 2D

La clase de mapeo 2D al igual que la anterior se ejecuta como un hilo independiente[10]. En cada posición de la malla, el hilo lanza un disparo usando `arm`, luego espera la respuesta mediante `read`, extrayendo la intensidad correspondiente al índice del pixel más cercano a la longitud de onda seleccionada. Dado que el desplazamiento de las etapas se hace de forma manual, el software muestra una barra de progreso que indica cuántas posiciones han sido adquiridas, permitiendo al operador saber en que celda se encuentra. Cuando finalmente se han completado todas las posiciones de la malla, se muestra el mapa 2D completo.

4.9. Módulos de interfaz gráfica de usuario

La interfaz gráfica se implementa en PyQt6[10] y se organiza mediante una serie de ventanas, que presenta cuatro modos principales:

1. **Pestaña de medida** (`MeasureTab`): muestra el espectro en tiempo real, siguiendo los parametros de adquisición.[7]
2. **Pestaña de comparación** (`CompareTab`): permite visualizar múltiples espectros, hacer zoom, usar cursores de selección de longitud de onda y comparar con espectros simulados del NIST.[9]
3. **Pestaña de perfil de profundidad** (`PlotTab`): muestra el valor de la intensidad para una longitud de onda seleccionada de todos los espectros en la pestaña de comparación.
4. **Pestaña de mapa 2D** (`PlotTab`): muestra el mapa de intensidad química generado por `MapThread`, renderizado con la función `imshow` de Matplotlib[13] y el mapa de color `plasma`.

Cuenta también con un panel lateral que nos permite modificar los parametros de adquisición y los controles de acción, separando de esta forma la vista de datos de los valores de configuración.

4.10. Módulo de almacenamiento

El sistema nos permite almacenar los datos de longitudes de ondas y espectros capturados en tres formatos:

- **TXT**: fichero de texto delimitado por tabuladores con cabecera, donde la primera columna contiene las longitudes de onda y las siguientes los espectros capturados.
- **ASCII**: formato similar al TXT pero con separación por espacios y cabecera de comentario (`#`).
- **Excel**: hoja de cálculo Excel con cabecera de dos filas (nombre del espectro y nombre de la sesión).

4.11. Flujo del procesamiento espectral

Cada espectro adquirido pasa por el siguiente flujo de procesamiento[11, 12] antes de ser visualizado o almacenado:

1. **Sustracción del mínimo:** $s \leftarrow s - \min(s)$. Corrección de la línea de base instrumental.
2. **Normalización:** $s \leftarrow s / \max(s)$. En caso de que se necesite el espectro es escalado al rango $[0, 1]$.
3. **Filtro de Savitzky-Golay**[8]: ventana de 11 puntos, polinomio de grado 3. Suaviza el ruido preservando la forma de los picos.

4.12. Visualización de espectros

La visualización de los espectros se realiza usando la librería matplotlib[13]. Se ha establecido un estilo visual oscuro (`facecolor #1a1a2e`, `#16213e`) con el objetivo de reducir la fatiga visual ante largas largas de laboratorio.

4.13. Pestaña de medida

Durante la adquisición, la pestaña de medida actualiza el gráfico tras cada disparo en tiempo real[10]. El modo *overlay* nos permite visualizar varios espectros superpuestos, facilitando la apreciación de variación entre disparos.

4.14. Pestaña comparativa

La pestaña comparativa nos permite gestionar la visualización de un conjunto de espectros[13]. Cada espectro puede ocultarse o eliminarse de forma individual. Para su identificación visual se asigna un color a cada espectro.

Esta pestaña cuenta con diversas herramientas para mejorar la interacción:

- **Cursor R / Cursor B:** nos permiten seleccionar dos longitudes de onda mediante

cursores en el espectro. El sistema calcula la diferencia $\Delta\lambda = \lambda_R - \lambda_B$, que puede emplearse como referencia para los procesos de perfil de profundidad y de mapeo 2D.

- **Zoom:** selecciona un area rectangular para ampliar la región espectral.
- **Reset:** restaura a la vista completa del espectro.

4.14.1. Simulación NIST

Esta ventana además incluye la posibilidad de superponer al espectro experimental un espectro simulado de un elemento de la tabla periódica[9]. Este espectro simulado posee los siguientes parametros:

- Temperatura electrónica T_e (en eV), rango 0.5–5.0 eV.
- Densidad electrónica N_e (en $\times 10^{17} \text{ cm}^{-3}$), rango 0.7–1.3.
- Porcentaje de concentración del elemento.
- Resolución espectral de la simulación.
- Rango de longitudes de onda.
- Nivel máximo de ionización.
- Modo de consulta NIST: estático (caché local) o dinámico (consulta en tiempo real a la base de datos NIST)[9].

4.15. Pestaña de perfil de profundidad

El módulo de perfil de profundidad permite analizar la variación de la intensidad espectral en función de la profundidad de la muestra[1], proporcionando una representación del comportamiento del material a distintas capas. El proceso consiste en:

1. El usuario selecciona la longitud de onda de interés (λ_R) en la pestaña comparativa.
2. Para cada espectro en la pestaña de comparación z_i se extrae la intensidad correspondiente a la longitud de onda seleccionada: $I(z_i) = s(\lambda_R)$.

3. Los valores obtenidos se representan en una gráfica intensidad–profundidad utilizando Matplotlib[13], permitiendo visualizar la evolución espectral a lo largo del perfil de profundidad.

4.16. Pestaña de mapas en 2D

El módulo de mapeo 2D genera una representación visual de la distribución espacial de un elemento en la superficie de la muestra. El proceso consiste en:

1. El usuario selecciona la longitud de onda de interés (λ_R) en la pestaña comparativa.
2. El sistema lanza **MapThread**, que adquiere un espectro por posición en una malla cuadrada de 5×5 puntos (configurable).
3. Para cada posición se extrae la intensidad en la longitud de onda λ_R : $I_{x,y} = s(\lambda_R)$.
4. El mapa resultante se representa mediante **imshow** de Matplotlib[13] con el mapa de color **plasma**, que proporciona una representación intuitiva de la intensidad.

La barra de progreso de la interfaz muestra el avance del proceso de mapeo en tiempo real[10].

Capítulo 5

Manual de usuario

5.1. Introducción

Esta sección describe una guía de uso del sistema, desde la conexión del espectrómetro hasta la exportación de los resultados. El objetivo es proporcionar al operador una guía de referencia que permita utilizar la aplicación sin necesidad de consultar el código fuente.

5.2. Instalación y requisitos previos

El sistema requiere un ordenador con Windows de 64 bits, Python 3.11 instalado y las bibliotecas recogidas en la tabla 1. Además, debe estar instalado el driver USB de Avantes, que se distribuye junto con la DLL `avaspec.dll` en el paquete de software del fabricante[7]. La instalación de las dependencias Python se realiza mediante el siguiente comando:

```
pip install PyQt6 matplotlib numpy scipy openpyxl SimulatedLIBS
```

Una vez instaladas las dependencias, la aplicación se lanza ejecutando el fichero principal:

```
python main.py
```

5.3. Conexión del espectrómetro

Al arrancar la aplicación, el espectrómetro debe estar conectado al ordenador mediante USB antes de pulsar el botón **Conectar** del panel lateral izquierdo.

Al iniciar la aplicación, la interfaz presenta la pestaña de medida activa por defecto, como se muestra en la figura 1. La columna izquierda agrupa los parámetros de adquisición y los controles de acción, mientras que el área central muestra la gráfica del espectro en tiempo real y el panel derecho informa del estado de la sesión y el contador de disparos. La barra inferior contiene los controles de almacenamiento y el campo de nombre de sesión.

El sistema ejecuta internamente la secuencia `AVS_Init` $\rightarrow$ `AVS_Activate` $\rightarrow$ `AVS_GetNumPixels` $\rightarrow$ `AVS_GetLambda`, obteniendo el vector de longitudes de onda calibradas del instrumento[7]. Si la conexión es exitosa, la barra de estado inferior muestra el número de canales detectados. En caso de error, se muestra un mensaje indicando que no se han encontrado dispositivos, lo que habitualmente indica que el driver USB no está instalado correctamente o que el cable no está conectado.

Para desconectar el espectrómetro de forma segura se debe pulsar el botón **Desconectar**, que detiene cualquier adquisición en curso antes de liberar el handle del dispositivo.

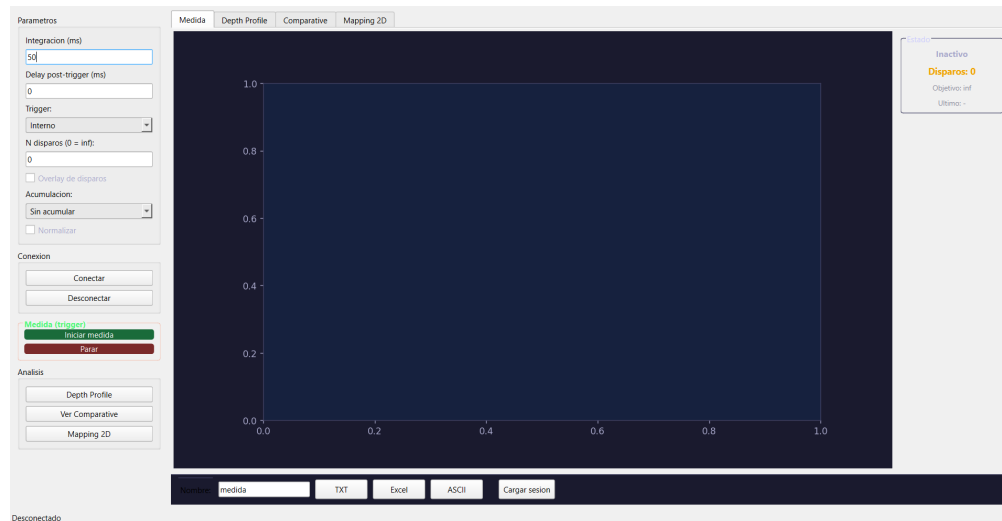


Figura 1. Interfaz gráfica del sistema.

5.4. Configuración de los parámetros de adquisición

El panel lateral izquierdo agrupa todos los parámetros configurables antes de iniciar una medida:

- **Integración (ms):** tiempo de exposición del detector por disparo. Valores típicos en

LIBS oscilan entre 1 ms y 100 ms según la intensidad del plasma y el material analizado. Un tiempo demasiado bajo produce espectros ruidosos, mientras que un tiempo excesivo puede saturar el detector.

- **Tiempo de retardo (ms)**: retardo entre el disparo hardware y el inicio de la lectura. Permite sincronizar la ventana de adquisición con la fase del plasma en la que las líneas espectrales son más nítidas, evitando la emisión continua inicial[1].
- **Trigger**: selecciona si el disparo se realiza mediante trigger interno o externo usando la señal TTL proveniente del láser.
- **Número de disparos**: número de espectros a adquirir en la sesión. El valor 0 activa el modo continuo, en el que la adquisición se prolonga hasta que el operador pulsa **Parar**.
- **Overlay de disparos**: si está activado, los espectros se superponen en la pestaña de medida en lugar de reemplazarse, lo que facilita la comparación visual entre disparos consecutivos.
- **Acumulación**: selecciona el modo de operación entre sin acumulación, suma acumulada o promedio acumulado.
- **Normalizar**: activa la normalización al rango $[0, 1]$ en el flujo de procesamiento.

5.5. Adquisición de espectros

Una vez configurados los parámetros, el operador pulsa **Iniciar medida**. La aplicación pasa a la pestaña de medida y muestra el estado *Esperando láser...*, indicando que el hilo **TriggerThread** está activo y a la espera del disparo hardware externo. El contador de disparos se actualiza en tiempo real con cada espectro recibido.

Cada espectro adquirido se procesa automáticamente (sustracción del mínimo, normalización opcional y filtro Savitzky-Golay) y se representa en la gráfica principal, con el eje horizontal en nanómetros y el eje vertical en intensidad. Al finalizar la sesión, ya sea por completar el número de disparos configurado o por pulsación manual del botón **Parar**, el sis-

tema realiza un autoguardado en formato TXT con el nombre base introducido en el campo **Nombre** de la barra inferior.

5.6. Comparación y análisis de espectros

Para acceder a la pestaña comparativa se pulsa el botón **Ver Comparative** del panel de análisis, o se hace clic directamente en la pestaña *Comparative*. En esta pestaña se muestran todos los espectros de la sesión actual superpuestos, cada uno con un color diferenciado. El panel derecho permite ocultar o eliminar espectros individuales mediante los controles de cada fila.

Las herramientas de interacción disponibles son:

- **Cursor R:** al activarlo y hacer clic sobre el espectro, fija el cursor rojo en la longitud de onda seleccionada (λ_R). Este valor se usa como referencia para el perfil de profundidad y el mapeo 2D.
- **Cursor B:** análogo al anterior, fija el cursor azul (λ_B). El sistema calcula y muestra automáticamente la diferencia $\Delta\lambda = \lambda_R - \lambda_B$.
- **Zoom:** permite seleccionar una región rectangular del espectro para ampliarla. Se activa el modo, se arrastra sobre la región de interés y se suelta.
- **Reset:** restaura la vista completa del espectro tras un zoom.

Para superponer un espectro simulado NIST, se selecciona el elemento en la lista del panel derecho, donde están ordenados de forma alfabética, se ajustan los parámetros del plasma (temperatura del electrón, densidad del electrón, resolución, rango de longitudes de onda y nivel de ionización) y se pulsa **Añadir simulación**. El espectro simulado aparece superpuesto al experimental con un color diferenciado, permitiendo identificar visualmente las líneas de emisión del elemento seleccionado[9].

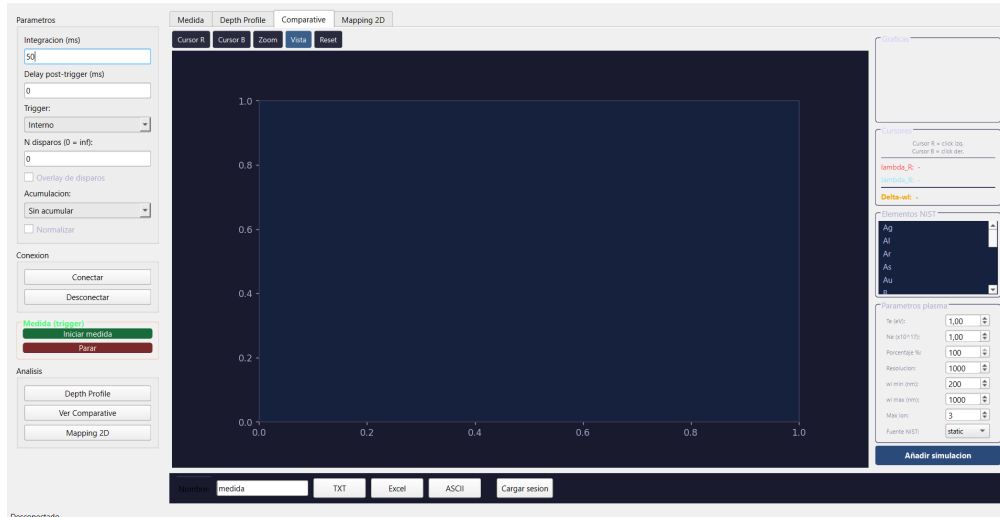


Figura 2. Interfaz gráfica de la pestaña de comparación.

5.7. Perfil de profundidad

Para generar un perfil de profundidad es necesario haber seleccionado previamente la longitud de onda de interés mediante el cursor λ_R en la pestaña comparativa. A continuación se pulsa el botón **Depth Profile** del panel de análisis. La aplicación extrae la intensidad en λ_R de cada espectro visible en la pestaña comparativa y representa los valores en la pestaña *Depth Profile*, con el número de disparo en el eje horizontal y la intensidad en el vertical. Cada punto del perfil corresponde a una capa progresivamente más profunda de la muestra, ya que cada disparo ablaciona una pequeña cantidad de material[1].

5.8. Mapeo 2D

El módulo de mapeo 2D requiere que el espectrómetro esté conectado y que se haya seleccionado la longitud de onda de interés mediante el cursor λ_R . Al pulsar **Mapping 2D**, la aplicación lanza el hilo **MapThread** y muestra la barra de progreso en el panel lateral. Recorriendo una malla de dimensiones 5x5.

El procedimiento que debe seguir el operador es el siguiente:

1. Posicionar la muestra en la primera celda de la malla (posición $[0, 0]$).
2. Esperar a que el sistema adquiera el espectro de esa posición y actualice la barra de

progreso.

3. Desplazar la muestra a la siguiente posición indicada y repetir el proceso hasta completar la malla.

Una vez adquiridas todas las posiciones, la pestaña *Mapping 2D* muestra el mapa de intensidad con el mapa de color **plasma**, donde los tonos más claros indican mayor intensidad del elemento analizado en esa posición de la superficie.

5.9. Almacenamiento y carga de sesiones

La barra inferior de la aplicación proporciona los controles de almacenamiento. El campo **Nombre** permite introducir un prefijo para los ficheros generados. Los botones **TXT**, **Excel** y **ASCII** abren un diálogo de guardado para exportar todos los espectros de la sesión actual en el formato seleccionado. La estructura de todos los formatos es compatible con la función **Cargar sesión**, que permite importar ficheros TXT de sesiones anteriores para compararlos con la sesión actual en la pestaña comparativa.

Capítulo 6

Resultados experimentales

Para validar el correcto funcionamiento del sistema software desarrollado, se han realizado varias pruebas mediante dos espectómetros, AvaSpec-Dual y AvaSpec de 3 canales[6], en el laboratorio UMALASERLAB:

- **Prueba de conexión:** se verificó que la aplicación detecta y activa correctamente los canales de ambos espectrómetros, obteniendo también el vector de longitud de onda correcto en cada caso[7].
- **Prueba de adquisición continua:** se comprobó que el hilo `TriggerThread` adquiere espectros de forma continua sin bloquear la interfaz gráfica[10], funcionando de forma fluida.
- **Prueba de procesamiento:** se verificó la correcta obtención de los espectros, comprobando la preservación de la forma de los picos [8] y comparando con espectros simulados[9].
- **Prueba de almacenamiento:** se exportaron sesiones en los tres formatos disponibles (TXT, ASCII, Excel) y se verificó la correcta recuperación de los datos mediante la función de carga de sesiones.
- **Prueba del perfil de profundidad:** se realizó una adquisición espectral sobre una muestra de referencia y se verificó que la variación de intensidad obtenida refleja el

comportamiento esperado del elemento seleccionado a distintas profundidades de la muestra.[1]

6.1. Ejemplos de espectros obtenidos

La figura 1 muestra la interfaz del software en la pestaña de medida, mostrando un espectro LIBS adquirido mediante el software desarrollado. Se puede ver que presenta un conjunto de líneas de emisión bien definidas sobre una línea de base que resulta casi nula, resultado del filtrado combinado a la sustracción del mínimo.

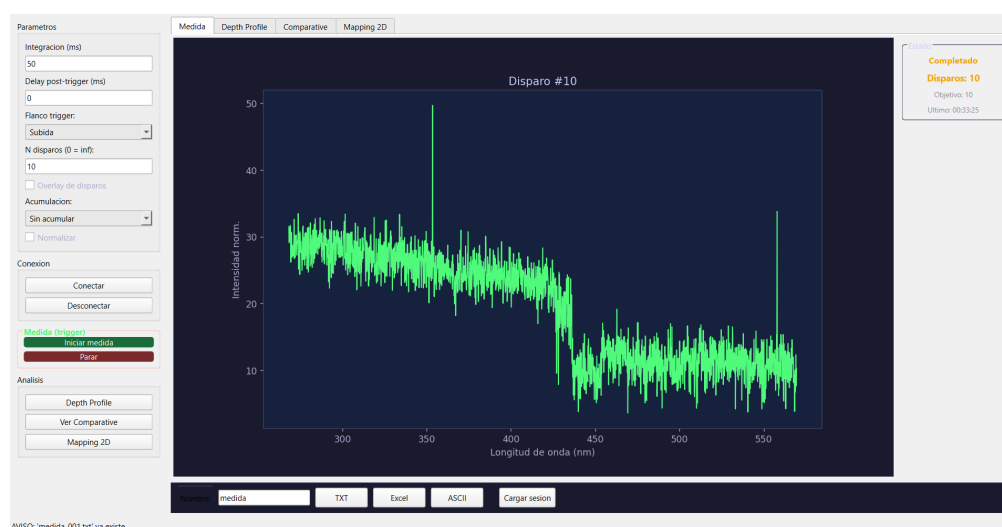


Figura 1. Ejemplo de espectro LIBS adquirido con el sistema desarrollado, con aplicación del filtro de Savitzky-Golay.

La figura 2 muestra la pestaña de comparación, enseñando varios espectros superpuestos que pertenecen a un conjunto de disparos. Se puede ver en la derecha como cada uno se puede ocultar o quitar. De esta forma se pueden detectar variaciones en los disparos.

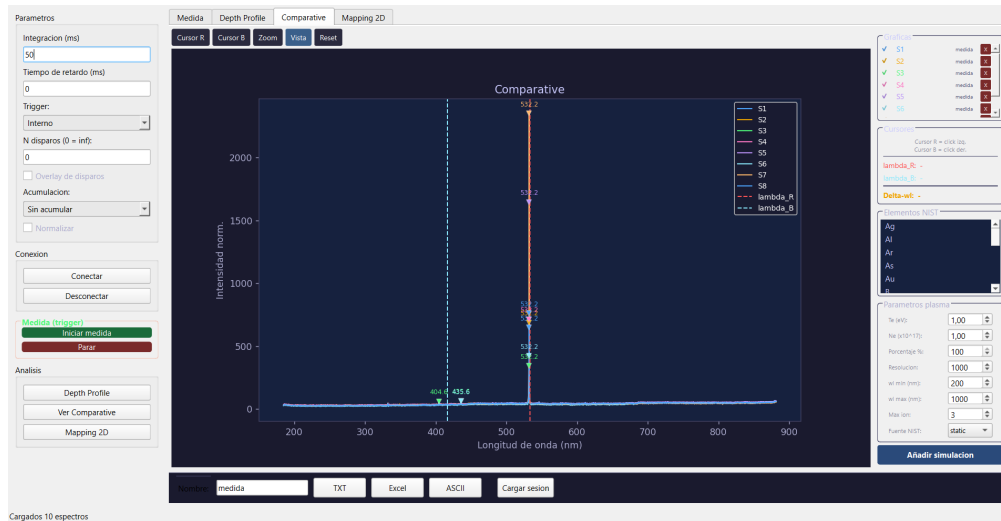


Figura 2. Vista de la pestaña comparativa con varios espectros superpuestos.

La figura 3 presenta la pestaña del perfil de profundidad obtenido de varias muestras sobre una longitud de onda seleccionada. Cada punto representa la intensidad de cada espectro. Pudiendo verse variaciones de intensidad debido a cambios en la concentración del elemento.

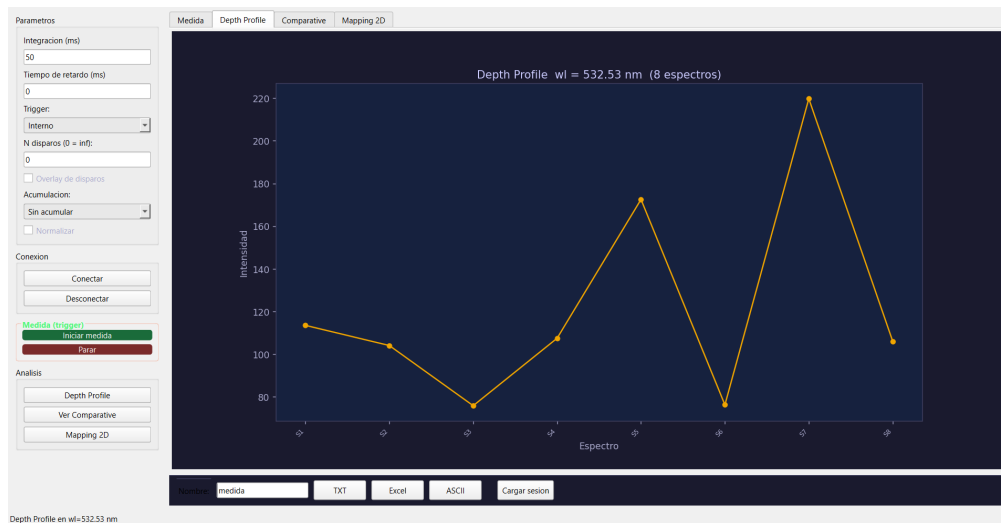


Figura 3. Perfil de profundidad para la longitud de onda de una línea espectral seleccionada.

La figura 4 muestra la pestaña de mapeo. Cada malla representa la intensidad adquirida en una posición espacial, codificada en forma de un color, los tonos claros representan una mayor concentración, respecto a los más oscuros.

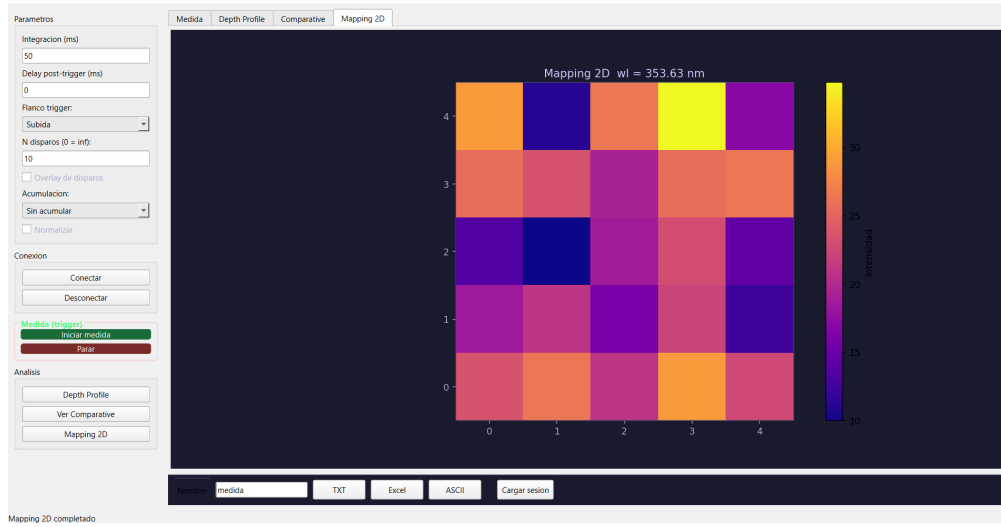


Figura 4. Mapa de intensidad química en 2D generado para la longitud de onda de una línea espectral seleccionada.

La figura 5 muestra la comparación de un espectro de cobre, cromo y aluminio simulados.

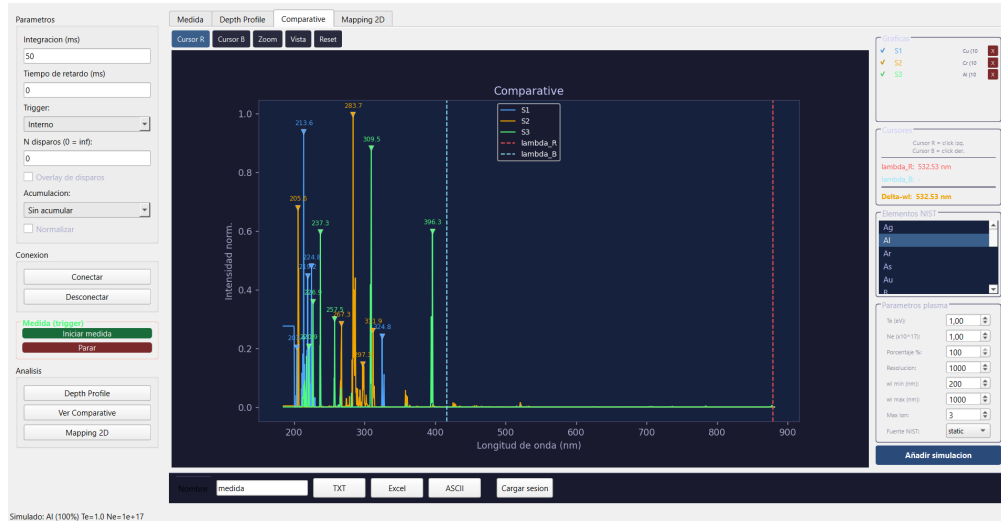


Figura 5. Espectros simulados con datos del NIST.

6.2. Evaluación del sistema

Los resultados obtenidos en las pruebas nos permiten observar que el sistema desarrollado cumple satisfactoriamente los requisitos funcionales planteados. Ya que contamos con una separación de lógica de adquisición en hilos independientes[10] podemos tener una interfaz gráfica fluida incluso en adquisiciones prolongadas.

El módulo de procesamiento espectral[11] nos ofrece espectros suavizados y normalizados de buena calidad. Mientras que la herramienta de comparación nos brinda una base de datos NIST[9] que facilita la identificación de los elementos de forma interactiva.

El módulo de mapeo 2D permite generar representaciones de la distribución espacial de los elementos de una forma semi-automática[4], permitiendo reducir el tiempo de análisis respecto al procedimiento manual previo.

En términos de flujo de trabajo, el sistema elimina la necesidad de exportar datos manualmente entre aplicaciones distintas para la adquisición y el análisis, reduciendo a un único software lo que anteriormente ocupaba el uso combinado de AvaSoft para la captura, junto con otros programas. Esto supone una simplificación significativa del proceso experimental, especialmente en sesiones con un número elevado de espectros.

Capítulo 7

Conclusiones

7.1. Conclusión

El objetivo principal de este Trabajo Fin de Grado era desarrollar un sistema software que automatizase el proceso de adquisición, almacenamiento y procesamiento inicial de datos espectrales obtenidos mediante un sistema LIBS. Tras el desarrollo, la integración con el hardware real y las pruebas realizadas en el laboratorio UMALASERLAB, puede afirmarse que dicho objetivo ha sido alcanzado satisfactoriamente.

Uno de los problemas de partida era la fragmentación del flujo de trabajo experimental. Como se ha ilustrado en el capítulo 2, ninguna de las soluciones comerciales existentes cubre de forma integrada las etapas de adquisición, procesamiento y análisis que requiere el grupo UMALASERLAB. El sistema desarrollado unifica todas estas etapas en una única aplicación de escritorio, permitiendo al investigador pasar de la adquisición del espectro a su análisis comparativo con la base de datos NIST o a la generación de un perfil de profundidad sin abandonar el entorno de la aplicación.

Desde el punto de vista técnico, la elección de Python como lenguaje de implementación ha resultado acertada. El acceso a la DLL de Avantes mediante `ctypes` ha permitido integrar el hardware sin necesidad de escribir extensiones nativas, manteniendo todo el código en un único lenguaje de alto nivel. Esto reduce significativamente la barrera de entrada para futuros desarrolladores del grupo, que no necesitan conocimientos de C para mantener o ampliar la aplicación. Las bibliotecas NumPy y SciPy han proporcionado las herramientas necesarias

para el procesamiento numérico de señales con un rendimiento más que suficiente para el caudal de datos generado por el espectrómetro, y PyQt6 ha facilitado la construcción de una interfaz gráfica estructurada y reactiva.

La arquitectura basada en el patrón MVC con hilos de trabajo independientes ha sido clave para garantizar la fluidez de la interfaz gráfica durante las adquisiciones. Al delegar las operaciones de lectura del espectrómetro a `TriggerThread` y `MapThread`, el hilo principal de la aplicación nunca queda bloqueado a la espera de datos del hardware. La comunicación entre hilos mediante señales y slots de Qt es segura frente a condiciones de carrera y se integra de forma natural con el bucle de eventos de la interfaz gráfica.

El desarrollo del proyecto ha puesto de manifiesto la complejidad inherente a la integración de hardware real en un sistema software. La sincronización entre el disparo del pulso láser y la ventana de adquisición del espectrómetro, la gestión de los tiempos de espera ante disparos no recibidos y la robustez ante fallos de comunicación con la DLL son aspectos que no resultan evidentes en una fase de diseño teórico y que han requerido un análisis cuidadoso del comportamiento del sistema en condiciones reales de laboratorio. En particular, la implementación de un mecanismo de *timeout* en la función `read` de la clase `Spectrometer` fue necesaria para evitar que la aplicación quedase bloqueada indefinidamente ante la ausencia de un disparo, situación habitual durante las fases de preparación del experimento.

El trabajo también ha puesto en valor la importancia de una estructura modular desde las primeras etapas del diseño. La separación clara entre la capa de hardware, la capa de procesamiento y la capa de presentación ha permitido desarrollar y depurar cada componente de forma independiente, reduciendo la propagación de errores entre módulos y facilitando la localización de fallos durante las pruebas de integración. Modificaciones en el algoritmo de procesamiento, como un cambio en los parámetros del filtro Savitzky-Golay, no tienen ningún efecto sobre el módulo de comunicación con el espectrómetro ni sobre los componentes de la interfaz gráfica, lo que hace que el código sea más mantenible y menos propenso a regresiones.

En conjunto, el sistema desarrollado supone una mejora real y tangible en el flujo de trabajo del equipo UMALASERLAB, reduciendo el tiempo dedicado a tareas instrumentales repetitivas y permitiendo al investigador centrarse en el análisis e interpretación de los resultados.

7.2. Futuras líneas de trabajo

A pesar de los resultados satisfactorios obtenidos, el software desarrollado constituye una base sólida sobre la que continuar construyendo. Durante el desarrollo y las pruebas han surgido varias líneas de mejora que se consideran de especial interés para aumentar las capacidades del sistema.

Integración del control de etapas motorizadas

La limitación más significativa del sistema en su estado actual es que el módulo de mapeo 2D requiere que el operador desplace manualmente la muestra entre posiciones. De forma paralela a este trabajo se ha desarrollado un sistema de control de etapas motorizadas XY que aún no ha sido integrado con la aplicación. La unión de ambos componentes permitiría que la adquisición del mapa 2D fuese completamente automática: el software enviaría las coordenadas de cada posición a las etapas, esperaría a que el posicionamiento fuese estable y lanzaría el disparo del láser, todo ello sin intervención del operador. Esta integración transformaría el módulo de mapeo en una herramienta de análisis elemental en superficie verdaderamente autónoma, con un impacto muy significativo sobre la productividad experimental del grupo.

Escalabilidad a otros modelos de espectrómetro

El sistema ha sido validado con dos espectrómetros de la familia Avantes: el AvaSpec-Dual y un AvaSpec de tres canales. Sin embargo, la clase **Spectrometer** está acoplada a la DLL de Avantes, lo que impide su uso directo con instrumentos de otros fabricantes. Sería conveniente definir una interfaz abstracta de espectrómetro que establezca el contrato de métodos que cualquier implementación concreta debe cumplir, de manera que añadir soporte para un nuevo modelo o fabricante se reduzca a implementar dicha interfaz sin modificar el resto del sistema. Esto aumentaría la portabilidad de la herramienta y permitiría su adopción por otros grupos de investigación con equipamiento diferente.

Identificación elemental automática

Actualmente la comparación con la base de datos NIST requiere que el usuario seleccione manualmente el elemento a simular. Una mejora de gran utilidad sería implementar un algoritmo de identificación automática que, a partir de los picos detectados en el espectro experimental, proponga los elementos más probablemente presentes ordenados por un índice de coincidencia espectral. El algoritmo podría basarse en la comparación de las posiciones de los picos detectados por `find_peaks` con las líneas de emisión tabuladas en NIST para cada elemento, ponderando la coincidencia por la intensidad relativa de cada línea. Esta funcionalidad reduciría considerablemente el tiempo de interpretación de los espectros, especialmente en materiales de composición desconocida o en análisis de rutina con un número elevado de muestras.

Análisis cuantitativo

El sistema actual es de carácter cualitativo: identifica la presencia de elementos pero no estima su concentración. Una extensión natural sería incorporar un módulo de calibración que permita construir curvas de calibración a partir de muestras de referencia de concentración conocida, y aplicarlas posteriormente para obtener estimaciones cuantitativas de la concentración elemental en muestras desconocidas. Esta capacidad es especialmente relevante en las aplicaciones industriales y medioambientales que desarrolla el grupo UMALASERLAB, donde no basta con identificar los elementos presentes sino que se requiere conocer su concentración con precisión.

Bibliografía

- [1] Miziolek, A. W., Palleschi, V., y Schechter, I., Laser-Induced Breakdown Spectroscopy (LIBS): Fundamentals and Applications. Cambridge University Press, 2006.
- [2] Singh, V. K., Tripathi, D. K., Deguchi, Y., y Wang, Z., Laser Induced Breakdown Spectroscopy (LIBS): Concepts, Instrumentation, Data Analysis and Applications. Wiley, 2023, doi:10.1002/9781119758396.
- [3] Anzano Lacarte, J., Lasheras, R. J., y Bello Gálvez, C., “Espectroscopía de descomposición inducida por láser (libs): una técnica emergente en la química analítica,” *Ciencia*, vol. 18, no. 3, 2010, <https://produccioncientificaluz.org/index.php/ciencia/article/view/9980>.
- [4] Noll, R., Fricke-Begemann, C., Connemann, S., Meinhardt, C., y Sturm, V., “Libs analyses for industrial applications – an overview of developments from 2014 to 2018,” *Journal of Analytical Atomic Spectrometry*, vol. 33, no. 6, pp. 945–956, 2018, doi:10.1039/C8JA00076J.
- [5] Harmon, R. S., DeLucia, F. C., McManus, C. E., McMillan, N. J., Jenkins, T. F., Walsh, M., y Miziolek, A., “Laser-induced breakdown spectroscopy – an emerging chemical sensor technology for real-time field-portable, geochemical, mineralogical, and environmental applications,” *Applied Geochemistry*, vol. 21, no. 5, pp. 730–747, 2006, doi:10.1016/j.apgeochem.2006.02.003.
- [6] BV, A., “Avaspec user manual,” 2023, <https://www.avantes.com>.
- [7] BV, A., AS5216x64 DLL Manual – Interface Package for 64 bit Windows Applications, 2012.

- [8] Savitzky, A. y Golay, M. J. E., “Smoothing and differentiation of data by simplified least squares procedures,” *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964, doi:10.1021/ac60214a047.
- [9] Kramida, A., Ralchenko, Y., Reader, J., y NIST ASD Team, “Nist atomic spectra database.” [Online]. National Institute of Standards and Technology, Gaithersburg, MD, 2024, doi:10.18434/T4W30F.
- [10] Limited, R. C., “Pyqt6 reference guide.” Online, 2021, <https://www.riverbankcomputing.com/static/Docs/PyQt6/>.
- [11] Virtanen, P., Gommers, R., Oliphant, T. E., *et al.*, “Scipy 1.0: Fundamental algorithms for scientific computing in python,” *Nature Methods*, vol. 17, pp. 261–272, 2020, doi:10.1038/s41592-019-0686-2.
- [12] Harris, C. R., Millman, K. J., van der Walt, S. J., *et al.*, “Array programming with numpy,” *Nature*, vol. 585, pp. 357–362, 2020, doi:10.1038/s41586-020-2649-2.
- [13] Hunter, J. D., “Matplotlib: A 2d graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi:10.1109/MCSE.2007.55.

Anexo 1

Código principal

Código 1.1. Librerías.

```
1 import ctypes
2 import time
3 import sys
4 import numpy as np
5 from PyQt6.QtWidgets import (
6     QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
7     QLabel, QPushButton, QLineEdit, QGroupBox, QFileDialog,
8     QStatusBar, QTabWidget, QProgressBar, QSizePolicy,
9     QCheckBox, QScrollArea, QFrame, QComboBox,
10    QDoubleSpinBox, QSpinBox, QListWidget, QAbstractItemView,
11    QMessageBox,
12 )
13 from PyQt6.QtCore import QThread, pyqtSignal, Qt
14 import matplotlib
15 matplotlib.use("QtAgg")
16 from matplotlib.backends.backend_qtagg import FigureCanvasQTagg as FigureCanvas
17 from matplotlib.figure import Figure
18 from matplotlib.patches import Rectangle as MplRect
19 from scipy.signal import savgol_filter, find_peaks
20
```

```

21 import avaspec1
22
23 COLORES = ["#4da6ff", "#f0a500", "#50fa7b", "#ff79c6", "#bd93f9", "#8be9fd", "#ffb86c
    ↪ "]
24 ELEMENTOS = [
25     "Ag","Al","Ar","As","Au","B","Ba","Be","Bi","Br","C","Ca","Cd","Ce","Cl",
26     "Co","Cr","Cs","Cu","Dy","Er","Eu","F","Fe","Ga","Gd","Ge","H","He","Hf",
27     "Hg","Ho","I","In","Ir","K","Kr","La","Li","Lu","Mg","Mn","Mo","N","Na",
28     "Nb","Nd","Ne","Ni","O","Os","P","Pb","Pd","Pr","Pt","Rb","Re","Rh","Ru",
29     "S","Sb","Sc","Se","Si","Sm","Sn","Sr","Ta","Tb","Tc","Te","Th","Ti","Tl",
30     "Tm","U","V","W","Xe","Y","Yb","Zn","Zr"
31 ]

```

Código 1.2. Capa de comunicación con el espectometro.

```

1 class Spectrometer:
2     def __init__(self, log, identity):
3         self.log = log
4         self.identity = identity
5         self.handle = None
6         self.connected = False
7         self.n_pixels = 0
8         self.wl = None
9         self.last_t = None
10
11     def connect(self):
12         self.handle = avaspec1.AVS_Activate(self.identity)
13         if self.handle <= 0:
14             self.log("Error al activar canal"); return False
15         pixels = ctypes.c_uint16()
16         avaspec1.AVS_GetNumPixels(self.handle, ctypes.byref(pixels))
17         self.n_pixels = pixels.value
18         wl_buf = (ctypes.c_double * self.n_pixels)()

```



```

19     avaspec1.AVS_GetLambda(self.handle, wl_buf)
20     self.wl = np.array(wl_buf[:self.n_pixels])
21     if np.max(self.wl) < 100:
22         self.wl = np.arange(self.n_pixels)
23     self.connected = True
24     return True
25
26 def disconnect(self):
27     try:
28         avaspec1.AVS_Deactivate(self.handle)
29     except Exception:
30         pass
31     self.connected = False
32
33 def prepare_hw(self, t_ms, edge=0):
34     meas = avaspec1.MeasConfigType()
35     meas.m_StartPixel      = 0
36     meas.m_StopPixel       = self.n_pixels - 1
37     meas.m_IntegrationTime = float(t_ms)
38     meas.m_NrAverages      = 1
39     meas.m_Trigger_m_Mode  = 1  # 0=SW interno, 1=HW externo
40     meas.m_Trigger_m_Source = 0
41     meas.m_Trigger_m_SourceType = edge
42     ret = avaspec1.AVS_PrepareMeasure(self.handle, meas)
43     self.last_t = None
44     self.log(f"AVS_PrepareMeasure ret={ret}")
45
46 def arm(self):
47     ret = avaspec1.AVS_Measure(self.handle, None, 1)
48     self.log(f"AVS_Measure ret={ret}")
49
50 def read(self, timeout=60):

```

```

51     raw  = (ctypes.c_double * self.n_pixels)()
52     ts   = ctypes.c_uint32()
53     start = time.time()
54     while not avaspec1.AVS_PollScan(self.handle):
55         if time.time() - start > timeout:
56             return None
57         time.sleep(0.002)
58     avaspec1.AVS_GetScopeData(self.handle, ctypes.byref(ts), raw)
59     return np.array(raw)

```

Código 1.3. Capa de procesamiento.

```

1 class Processor:
2     @staticmethod
3     def process(s):
4         s = s - np.min(s)
5         if np.max(s) > 0:
6             s = s / np.max(s)
7         return savgol_filter(s, 11, 3)

```

Código 1.4. Hilo para la medición.

```

1 class TriggerThread(QThread):
2     shot_received = pyqtSignal(int, object, object)
3     finished_all  = pyqtSignal()
4
5     def __init__(self, specs, t_ms, n_shots=0, edge=0,
6                 normalize=True, accum_mode=0, delay_ref=None):
7         super().__init__()
8         self.specs      = specs
9         self.t_ms       = t_ms
10        self.n_shots     = n_shots
11        self.edge        = edge

```

```

12     self.normalize = normalize
13     self.accum_mode = accum_mode
14     self.delay_ref = delay_ref
15     self.__running = True
16     self.__shot_idx = 0
17     self.__accum = None
18
19     def run(self):
20         for spec in self.specs:
21             spec.prepare_hw(self.t_ms, self.edge)
22
23         while self.__running:
24             if self.n_shots > 0 and self.__shot_idx >= self.n_shots:
25                 break
26
27             for spec in self.specs:
28                 spec.arm()
29
30             if self.delay_ref is not None:
31                 try:
32                     delay = float(self.delay_ref.text())/1000
33                 except ValueError:
34                     delay = 0.0
35             else:
36                 delay = 0.0
37             if delay > 0:
38                 time.sleep(delay)
39
40             spectra, wavelengths = [], []
41             for spec in self.specs:
42                 s = spec.read(timeout=60)
43                 if s is None:

```

```

44         continue
45         s = Processor.process(s) if self.normalize else s - np.min(s)
46         spectra.append(s)
47         wavelengths.append(spec.wl)
48
49     if not spectra:
50         continue
51
52     wl_min = min(w[0] for w in wavelengths)
53     wl_max = max(w[-1] for w in wavelengths)
54     wl_c = np.linspace(wl_min, wl_max, 2000)
55     si = [np.interp(wl_c, wl, s) for wl, s in zip(wavelengths, spectra)]
56     s_out = np.mean(si, axis=0)
57
58     if self.accum_mode == 1:
59         self._accum = s_out.copy() if self._accum is None \
60             else self._accum + s_out
61         emit_s = self._accum
62     elif self.accum_mode == 2:
63         n = self._shot_idx + 1
64         self._accum = s_out.copy() if self._accum is None \
65             else self._accum * (n - 1) / n + s_out / n
66         emit_s = self._accum
67     else:
68         emit_s = s_out
69
70     self._shot_idx += 1
71     self.shot_received.emit(self._shot_idx, wl_c, emit_s)
72
73     self.finished_all.emit()
74
75     def stop(self):

```

```
76     self.__running = False
```

Código 1.5. Hilo para el mapeado.

```
1 class MapThread(QThread):
2     progress = pyqtSignal(int)
3     done     = pyqtSignal(object)
4
5     def __init__(self, specs, t_ms, wl, selected_wl, grid=5, edge=0):
6         super().__init__()
7         self.specs      = specs
8         self.t_ms       = t_ms
9         self.wl         = wl
10        self.selected_wl = selected_wl
11        self.grid        = grid
12        self.edge        = edge
13
14    def run(self):
15        idx = (np.abs(self.wl - self.selected_wl)).argmin()
16        cube = np.zeros((self.grid, self.grid))
17        total = self.grid ** 2
18        for spec in self.specs:
19            spec.prepare_hw(self.t_ms, self.edge)
20        k = 0
21        for y in range(self.grid):
22            for x in range(self.grid):
23                for spec in self.specs:
24                    spec.arm()
25                    s = self.specs[0].read(timeout=60)
26                    if s is not None:
27                        cube[y, x] = float((s - np.min(s))[idx])
28                    k += 1
29        self.progress.emit(int(k / total * 100))
```

```
30     self.done.emit(cube)
```

Código 1.6. Módulo de gráficas.

```
1 class PlotTab(QWidget):
2     def __init__(self, parent=None):
3         super().__init__(parent)
4         layout = QVBoxLayout(self)
5         layout.setContentsMargins(4, 4, 4, 4)
6         self.fig = Figure(facecolor="#1a1a2e")
7         self.ax = self.fig.add_subplot(111)
8         self._style_ax()
9         self.canvas = FigureCanvas(self.fig)
10        self.canvas.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Expanding)
11        layout.addWidget(self.canvas)
12
13    def _style_ax(self):
14        self.ax.set_facecolor("#16213e")
15        self.ax.tick_params(colors="#aaaacc")
16        for sp in self.ax.spines.values(): sp.set_color("#444466")
17        self.ax.title.set_color("#ccccff")
18        self.ax.xaxis.label.set_color("#aaaacc")
19        self.ax.yaxis.label.set_color("#aaaacc")
```

Código 1.7. Módulo de la pestaña de medida.

```
1 class MeasureTab(QWidget):
2     def __init__(self, parent=None):
3         super().__init__(parent)
4         outer = QHBoxLayout(self)
5         outer.setContentsMargins(4, 4, 4, 4)
6         outer.setSpacing(8)
7
```

```

8     self.fig = Figure(facecolor="#1a1a2e")
9     self.ax = self.fig.add_subplot(111)
10    self.__style_ax()
11    self.canvas = FigureCanvas(self.fig)
12    self.canvas.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Expanding)
13    outer.addWidget(self.canvas, stretch=1)
14
15    panel = QWidget(); panel.setFixedWidth(165)
16    pl = QVBoxLayout(panel)
17    pl.setContentsMargins(4, 4, 4, 4); pl.setSpacing(8)
18    outer.addWidget(panel)
19
20    g = QGroupBox("Estado")
21    g.setStyleSheet("QGroupBox{color:#ccccff;border:1px solid #444466;"
22                   "border-radius:4px;margin-top:6px}"
23                   "QGroupBox::title{subcontrol-origin:margin;left:8px}")
24    gl = QVBoxLayout(g); gl.setContentsMargins(8, 14, 8, 8); gl.setSpacing(8)
25
26    self.lbl_state = QLabel("Inactivo")
27    self.lbl_state.setStyleSheet("color:#aaaacc;font-size:13px;font-weight:bold")
28    self.lbl_state.setAlignment(Qt.AlignmentFlag.AlignCenter)
29    self.lbl_shots = QLabel("Disparos: 0")
30    self.lbl_shots.setStyleSheet("color:#f0a500;font-size:14px;font-weight:bold")
31    self.lbl_shots.setAlignment(Qt.AlignmentFlag.AlignCenter)
32    self.lbl_target = QLabel("Objetivo: inf")
33    self.lbl_target.setStyleSheet("color:#888899;font-size:11px")
34    self.lbl_target.setAlignment(Qt.AlignmentFlag.AlignCenter)
35    self.lbl_time = QLabel("Ultimo: -")
36    self.lbl_time.setStyleSheet("color:#888899;font-size:11px")
37    self.lbl_time.setAlignment(Qt.AlignmentFlag.AlignCenter)
38
39    gl.addWidget(self.lbl_state); gl.addWidget(self.lbl_shots)

```

```

40     gl.addWidget(self.lbl_target); gl.addWidget(self.lbl_time)
41     pl.addWidget(g); pl.addStretch()
42     self.__overlay_lines = []
43
44     def __style_ax(self):
45         self.ax.set_facecolor("#16213e")
46         self.ax.tick_params(colors="#aaaacc")
47         for sp in self.ax.spines.values(): sp.set_color("#444466")
48         self.ax.title.set_color("#ccccff")
49         self.ax.xaxis.label.set_color("#aaaacc")
50         self.ax.yaxis.label.set_color("#aaaacc")
51
52     def set_state(self, text, color="#aaaacc"):
53         self.lbl_state.setText(text)
54         self.lbl_state.setStyleSheet(f"color:{color};font-size:13px;font-weight:bold")
55
56     def set_target(self, n):
57         self.lbl_target.setText(f"Objetivo: {'inf' if n == 0 else n}")
58
59     def update_shot(self, idx, wl, s, overlay=False, max_overlay=10):
60         self.lbl_shots.setText(f"Disparos: {idx}")
61         self.lbl_time.setText(f"Ultimo: {time.strftime('%H: %M: %S')}")
62         if overlay:
63             alphas = np.linspace(0.12, 1.0, max_overlay)
64             color = COLORS[(idx - 1) % len(COLORS)]
65             line, = self.ax.plot(wl, s, color=color, linewidth=1, alpha=1.0)
66             self.__overlay_lines.append(line)
67             if len(self.__overlay_lines) > max_overlay:
68                 self.__overlay_lines.pop(0).remove()
69             for i, ln in enumerate(self.__overlay_lines):
70                 ln.set_alpha(alphas[i + max_overlay - len(self.__overlay_lines)])
71         else:

```



```

72         self.ax.clear(); self.__style_ax()
73         self.ax.plot(wl, s, color="#50fa7b", linewidth=1.2)
74         self.ax.set_xlabel("Longitud de onda (nm)")
75         self.ax.set_ylabel("Intensidad norm.")
76         self.ax.set_title(f"Disparo #{idx}")
77         self.canvas.draw_idle()
78
79     def reset(self):
80         self.__overlay_lines = []
81         self.ax.clear(); self.__style_ax()
82         self.canvas.draw()
83         self.lbl_shots.setText("Disparos: 0")
84         self.lbl_time.setText("Ultimo: -")

```

Código 1.8. Módulo para la pestaña de comparación.

```

1 class CompareTab(QWidget):
2     add_sim_requested = pyqtSignal()
3
4     def __init__(self, parent=None):
5         super().__init__(parent)
6         self._mode = "view"
7         self._zoom_start = None
8         self._zoom_patch = None
9         self._zoom_cids = []
10
11         outer = QHBoxLayout(self)
12         outer.setContentsMargins(4, 4, 4, 4); outer.setSpacing(6)
13
14         left = QWidget()
15         lv = QVBoxLayout(left); lv.setContentsMargins(0, 0, 0, 0); lv.setSpacing(4)
16         outer.addWidget(left, stretch=1)
17

```

```

18     tb = QHBoxLayout(); tb.setSpacing(4)
19     self._tb_btns = {}
20     for key, label, tip in [
21         ("cursor_r", "Cursor R", "Click fija cursor rojo (lambda_R)"),
22         ("cursor_b", "Cursor B", "Click fija cursor azul (lambda_B)"),
23         ("zoom", "Zoom", "Arrastra para hacer zoom en region"),
24         ("view", "Vista", "Solo visionado"),
25         ("reset", "Reset", "Volver a vista original"),
26     ]:
27         btn = QPushButton(label)
28         btn.setToolTip(tip)
29         btn.setFixedHeight(26)
30         btn.setStyleSheet(self._btn_style(key == "view"))
31         if key == "reset":
32             btn.clicked.connect(self._reset_zoom)
33         else:
34             btn.clicked.connect(lambda _c, k=key: self._set_mode(k))
35         self._tb_btns[key] = btn
36         tb.addWidget(btn)
37     tb.addStretch()
38     lv.addLayout(tb)
39
40     self.fig = Figure(facecolor="#1a1a2e")
41     self.ax = self.fig.add_subplot(111)
42     self._style_ax()
43     self.canvas = FigureCanvas(self.fig)
44     self.canvas.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Expanding)
45     lv.addWidget(self.canvas)
46
47     right = QWidget(); right.setFixedWidth(200)
48     rl = QVBoxLayout(right); rl.setContentsMargins(4, 4, 4, 4); rl.setSpacing(4)
49     outer.addWidget(right)

```

```

50
51 g_spec = QGroupBox("Graficas")
52 g_spec.setStyleSheet("QGroupBox{color:#ccccff;border:1px solid #444466;"
53                       "border-radius:4px;margin-top:6px}"
54                       "QGroupBox::title{subcontrol-origin:margin;left:8px}")
55 gsl = QVBoxLayout(g_spec); gsl.setContentsMargins(4, 12, 4, 4); gsl.setSpacing(2)
56 scroll_spec = QScrollArea(); scroll_spec.setWidgetResizable(True)
57 scroll_spec.setFrameShape(QFrame.Shape.NoFrame)
58 scroll_spec.setStyleSheet("background:transparent")
59 self._cb_container = QWidget()
60 self._cb_layout    = QVBoxLayout(self._cb_container)
61 self._cb_layout.setContentsMargins(0, 0, 0, 0); self._cb_layout.setSpacing(2)
62 self._cb_layout.addStretch()
63 scroll_spec.setWidget(self._cb_container)
64 gsl.addWidget(scroll_spec)
65 rl.addWidget(g_spec, stretch=2)
66
67 # --- 2. Cursores ---
68 g_cur = QGroupBox("Cursores")
69 g_cur.setStyleSheet("QGroupBox{color:#ccccff;border:1px solid #444466;"
70                    "border-radius:4px;margin-top:6px}"
71                    "QGroupBox::title{subcontrol-origin:margin;left:8px}")
72 gcl = QVBoxLayout(g_cur); gcl.setContentsMargins(8, 14, 8, 6); gcl.setSpacing(4)
73 hint = QLabel("Cursor R = click izq.\nCursor B = click der.")
74 hint.setStyleSheet("color:#888899;font-size:9px")
75 hint.setAlignment(Qt.AlignmentFlag.AlignCenter)
76 gcl.addWidget(hint)
77 sep = QFrame(); sep.setFrameShape(QFrame.Shape.HLine); sep.setStyleSheet("color
↪ :#444466")
78 gcl.addWidget(sep)
79 self.lbl_red   = self._clabel("lambda_R:", "#ff5555")
80 self.lbl_blue  = self._clabel("lambda_B:", "#8be9fd")

```

```

81     self.lbl_delta = self.__clabel("Delta-wl:", "#f0a500", bold=True)
82     gcl.addWidget(self.lbl_red); gcl.addWidget(self.lbl_blue)
83     sep2 = QFrame(); sep2.setFrameShape(QFrame.Shape.HLine); sep2.setStyleSheet("
↪ color:#444466")
84     gcl.addWidget(sep2); gcl.addWidget(self.lbl_delta)
85     rl.addWidget(g_cur)
86
87     # --- 3. Lista de elementos NIST ---
88     g_elem = QGroupBox("Elementos NIST")
89     g_elem.setStyleSheet("QGroupBox{color:#aaaacc;border:1px solid #444466;"
90                           "border-radius:4px;margin-top:6px}"
91                           "QGroupBox::title{subcontrol-origin:margin;left:8px}")
92     gel = QVBoxLayout(g_elem); gel.setContentsMargins(4, 12, 4, 4); gel.setSpacing(2)
93     self.elem_list = QListWidget()
94     self.elem_list.setStyleSheet(
95         "QListWidget{background:#16213e;border:none;}"
96         "QListWidget::item{color:#ccccff;padding:2px 4px;font-size:11px;}"
97         "QListWidget::item:selected{background:#3a5f8a;color:white;}"
98         "QListWidget::item:hover{background:#2a2a5e;}"
99     )
100     for el in ELEMENTS:
101         self.elem_list.addItem(el)
102     self.elem_list.setSelectionMode(QAbstractItemView.SelectionMode.SingleSelection)
103     gel.addWidget(self.elem_list)
104     rl.addWidget(g_elem, stretch=2)
105
106     # --- 4. Parametros del plasma ---
107     g_par = QGroupBox("Parametros plasma")
108     g_par.setStyleSheet("QGroupBox{color:#aaaacc;border:1px solid #444466;"
109                           "border-radius:4px;margin-top:6px}"
110                           "QGroupBox::title{subcontrol-origin:margin;left:8px}")
111     gpl = QVBoxLayout(g_par); gpl.setContentsMargins(6, 14, 6, 6); gpl.setSpacing(3)

```

```

112
113     def prow(lbl_txt, widget):
114         r = QHBoxLayout(); r.setSpacing(4)
115         l = QLabel(lbl_txt); l.setStyleSheet("color:#888899;font-size:9px;")
116         l.setFixedWidth(76)
117         r.addWidget(l); r.addWidget(widget)
118         return r
119
120     self.sim_Te = QDoubleSpinBox()
121     self.sim_Te.setRange(0.5, 5.0); self.sim_Te.setValue(1.0)
122     self.sim_Te.setSingleStep(0.1); self.sim_Te.setDecimals(2); self.sim_Te.setFixedWidth
↪ (68)
123
124     self.sim_Ne = QDoubleSpinBox()
125     self.sim_Ne.setRange(0.7, 1.3); self.sim_Ne.setValue(1.0)
126     self.sim_Ne.setSingleStep(0.05); self.sim_Ne.setDecimals(2); self.sim_Ne.
↪ setFixedWidth(68)
127     self.sim_Ne.setToolTip("x1017 cm-3")
128
129     self.sim_pct = QSpinBox()
130     self.sim_pct.setRange(1, 100); self.sim_pct.setValue(100)
131     self.sim_pct.setFixedWidth(68)
132     self.sim_pct.setToolTip("Porcentaje del elemento seleccionado")
133
134     self.sim_res = QSpinBox()
135     self.sim_res.setRange(500, 10000); self.sim_res.setValue(1000)
136     self.sim_res.setSingleStep(100); self.sim_res.setFixedWidth(68)
137
138     self.sim_low = QSpinBox()
139     self.sim_low.setRange(100, 900); self.sim_low.setValue(200)
140     self.sim_low.setSingleStep(50); self.sim_low.setFixedWidth(68)
141

```

```

142     self.sim_up = QSpinBox()
143     self.sim_up.setRange(200, 1100); self.sim_up.setValue(1000)
144     self.sim_up.setSingleStep(50); self.sim_up.setFixedWidth(68)
145
146     self.sim_ion = QSpinBox()
147     self.sim_ion.setRange(1, 5); self.sim_ion.setValue(3); self.sim_ion.setFixedWidth(68)
148
149     self.sim_ws = QComboBox()
150     self.sim_ws.addItem("static", "dynamic"); self.sim_ws.setFixedWidth(68)
151     self.sim_ws.setStyleSheet("QComboBox{color:black;background:#e8e8e8;font-size:10px
↵ ;}")
152     self.sim_ws.setToolTip("static=cache local  dynamic=NIST en vivo")
153
154     gpl.addLayout(prow("Te (eV):", self.sim_Te))
155     gpl.addLayout(prow("Ne (x10^17):", self.sim_Ne))
156     gpl.addLayout(prow("Porcentaje %:", self.sim_pct))
157     gpl.addLayout(prow("Resolucion:", self.sim_res))
158     gpl.addLayout(prow("wl min (nm):", self.sim_low))
159     gpl.addLayout(prow("wl max (nm):", self.sim_up))
160     gpl.addLayout(prow("Max ion:", self.sim_ion))
161     gpl.addLayout(prow("Fuente NIST:", self.sim_ws))
162     rl.addWidget(g_par)
163
164     # --- 5. Boton Añadir ---
165     self.btn_add_sim = QPushButton("Añadir simulacion")
166     self.btn_add_sim.setFixedHeight(30)
167     self.btn_add_sim.setStyleSheet(
168         "QPushButton{background:#2a4a7a;color:white;border-radius:4px;font-weight:bold
↵ ;}"
169         "QPushButton:hover{background:#3a6a9a;}"
170     )
171     self.btn_add_sim.clicked.connect(self.add_sim_requested)

```

```

172         rl.addWidget(self.btn_add_sim)
173
174         # estado interno
175         self.checkboxes = []
176         self._cursor_r = self._cursor_b = None
177         self._wl_r = self._wl_b = None
178
179     def _btn_style(self, active=False):
180         bg = "#3a5f8a" if active else "#2a2a3e"
181         return (f"QPushButton{{{background:{bg};color:white;border-radius:3px;"
182                 f"font-size:11px;padding:2px 8px;}}}"
183                 f"QPushButton:hover{{{background:#4a6f9a;}}}")
184
185     def _set_mode(self, mode):
186         self._mode = mode
187         for key, btn in self._tb_btns.items():
188             if key == "reset": continue
189             btn.setStyleSheet(self._btn_style(key == mode))
190
191     def _reset_zoom(self):
192         self.ax.autoscale(); self.canvas.draw_idle()
193
194     def _clabel(self, prefix, color, bold=False):
195         lbl = QLabel(f"{{{prefix}} -")
196         lbl.setStyleSheet(f"color:{color};font-size:11px;"
197                            f"font-weight:{{{bold}} if bold else 'normal'}")
198         return lbl
199
200     def _style_ax(self):
201         self.ax.set_facecolor("#16213e")
202         self.ax.tick_params(colors="#aaaacc")
203         for sp in self.ax.spines.values(): sp.set_color("#444466")

```

```

204     self.ax.title.set_color("#ccccff")
205     self.ax.xaxis.label.set_color("#aaaacc")
206     self.ax.yaxis.label.set_color("#aaaacc")
207
208 def add_spectrum_row(self, display_name, session_label, color, on_change, on_delete):
209     row = QWidget()
210     rl = QHBoxLayout(row); rl.setContentsMargins(0, 0, 0, 0); rl.setSpacing(2)
211
212     cb = QCheckBox(f" {display_name}")
213     cb.setToolTip(f"Conjunto: {session_label}")
214     cb.setStyleSheet(f"QCheckBox{{{color:{color}};font-size:11px}}"
215                      "QCheckBox::indicator{width:12px;height:12px}")
216     cb.setChecked(True)
217     cb.stateChanged.connect(on_change)
218     rl.addWidget(cb, stretch=1)
219
220     set_lbl = QLabel(session_label[:6])
221     set_lbl.setStyleSheet("color:#555577;font-size:8px;")
222     set_lbl.setFixedWidth(36)
223     rl.addWidget(set_lbl)
224
225     del_btn = QPushButton("X")
226     del_btn.setFixedSize(16, 16)
227     del_btn.setStyleSheet(
228         "QPushButton{background:#7a2a2a;color:white;border-radius:2px;"
229         "font-size:8px;padding:0;}"
230         "QPushButton:hover{background:#aa3333;}"
231     )
232     del_btn.clicked.connect(on_delete)
233     rl.addWidget(del_btn)
234
235     item = self._cb_layout.takeAt(self._cb_layout.count() - 1)

```



```

236     self.__cb_layout.addWidget(row)
237     self.__cb_layout.addItem(item)
238     self.checkboxes.append((cb, row, set_lbl))
239
240 def update_row_name(self, idx, display_name, color):
241     cb, __, __ = self.checkboxes[idx]
242     cb.setText(f" {display_name}")
243     cb.setStyleSheet(f"QCheckBox{{{color}:{color};font-size:11px}}}"
244                     "QCheckBox::indicator{width:12px;height:12px}")
245
246 def visible_indices(self):
247     return [i for i, (cb, __, __) in enumerate(self.checkboxes) if cb.isChecked()]
248
249 def clear_spectra(self):
250     for __, row, __ in self.checkboxes:
251         row.deleteLater()
252     self.checkboxes = []
253     while self.__cb_layout.count() > 1:
254         item = self.__cb_layout.takeAt(0)
255         if item.widget():
256             item.widget().deleteLater()
257     self.__wl_r = self.__wl_b = None
258     self.__cursor_r = self.__cursor_b = None
259     self.__zoom_start = None
260     self.__zoom_patch = None
261     self.ax.clear(); self.__style_ax()
262     self.canvas.draw()
263
264 def update_cursor_display(self):
265     r = f"{self.__wl_r:.2f} nm" if self.__wl_r is not None else "-"
266     b = f"{self.__wl_b:.2f} nm" if self.__wl_b is not None else "-"
267     self.lbl_red.setText(f"lambda_R: {r}")

```

```

268     self.lbl_blue.setText(f"lambda_B: {b}")
269     if self._wl_r is not None and self._wl_b is not None:
270         d = self._wl_r - self._wl_b
271         self.lbl_delta.setText(f"Delta-wl: {'+' if d >= 0 else ''}{d:.2f} nm")
272     elif self._wl_r is not None:
273         self.lbl_delta.setText(f"Delta-wl: {self._wl_r:.2f} nm")
274     else:
275         self.lbl_delta.setText("Delta-wl: -")

```

Código 1.9. Aplicación principal.

```

1 class App(QMainWindow):
2     def __init__(self):
3         super().__init__()
4         self.specs      = []
5         self.wl          = None
6         self.last_spectrum = None
7         self.saved       = []
8         self.selected_wl  = None
9         self._trig_thread = None
10        self._map_thread  = None
11        self._session_count = 0
12        self._peak_cache   = {}
13
14        self.setWindowTitle("LIBS PRO - PyQt6")
15        self.resize(1350, 780)
16        self._build_ui()
17
18    def log(self, msg):
19        print(msg)
20        self.statusBar().showMessage(msg)
21
22    def connect_devices(self):

```

```

23     n = avaspec1.AVS_GetNrOfDevices()
24     if n <= 0: self.log("No hay dispositivos"); return
25     ids = (avaspec1.AvsIdentityType * n)()
26     req = ctypes.c_int()
27     avaspec1.AVS_GetList(ctypes.sizeof(ids), ctypes.byref(req), ids)
28     self.specs = []
29     for i in range(n):
30         spec = Spectrometer(self.log, ids[i])
31         if spec.connect(): self.specs.append(spec)
32     self.log(f"{len(self.specs)} canal(es) conectados" if self.specs else "Error al conectar")
33
34 def disconnect__devices(self):
35     self.stop()
36     for s in self.specs: s.disconnect()
37     self.specs = []
38     self.log("Desconectado")
39
40 def __build__ui(self):
41     root = QWidget(); self.setCentralWidget(root)
42     rl = QHBoxLayout(root); rl.setSpacing(10)
43
44     left = QWidget(); left.setFixedWidth(230)
45     ll = QVBoxLayout(left); ll.setSpacing(8); ll.setContentsMargins(6, 6, 6, 6)
46     rl.addWidget(left)
47
48     g1 = QGroupBox("Parametros")
49     v1 = QVBoxLayout(g1)
50     v1.addWidget(QLabel("Integracion (ms)"))
51     self.t = QLineEdit("50"); v1.addWidget(self.t)
52     v1.addWidget(QLabel("Tiempo de retardo (ms)"))
53     self.t_re = QLineEdit("0"); v1.addWidget(self.t_re)
54     v1.addWidget(QLabel("Trigger:"))

```

```

55     self.cmb_edge = QComboBox()
56     self.cmb_edge.addItem("Interno", "Externo")
57     self.cmb_edge.setStyleSheet(
58         "QComboBox{color:black;background:#e8e8e8;border:1px solid #aaa;"
59         "border-radius:3px;padding:2px 6px;}"
60         "QComboBox QAbstractItemView{color:black;background:#f0f0f0;}")
61     v1.addWidget(self.cmb_edge)
62     v1.addWidget(QLabel("N disparos (0 = inf):"))
63     self.spin_shots = QLineEdit("0"); v1.addWidget(self.spin_shots)
64     self.chk_overlay = QCheckBox("Overlay de disparos")
65     self.chk_overlay.setChecked(False)
66     self.chk_overlay.setStyleSheet("color:#aaaacc"); v1.addWidget(self.chk_overlay)
67     v1.addWidget(QLabel("Acumulacion:"))
68     self.cmb_accum = QComboBox()
69     self.cmb_accum.addItem("Sin acumular", "Suma acumulada", "Promedio")
70     self.cmb_accum.setStyleSheet(
71         "QComboBox{color:black;background:#e8e8e8;border:1px solid #aaa;"
72         "border-radius:3px;padding:2px 6px;}"
73         "QComboBox QAbstractItemView{color:black;background:#f0f0f0;}")
74     v1.addWidget(self.cmb_accum)
75     self.chk_normalize = QCheckBox("Normalizar")
76     self.chk_normalize.setChecked(False)
77     self.chk_normalize.setStyleSheet("color:#aaaacc"); v1.addWidget(self.chk_normalize)
78     ll.addWidget(g1)
79
80     g2 = QGroupBox("Conexion")
81     v2 = QVBoxLayout(g2)
82     self._btn(v2, "Conectar", self.connect_devices)
83     self._btn(v2, "Desconectar", self.disconnect_devices)
84     ll.addWidget(g2)
85
86     g3 = QGroupBox("Medida (trigger)")

```

```

87     g3.setStyleSheet("QGroupBox{color:#50fa7b;font-weight:bold;"
88                       "border:1px solid #50fa7b55;border-radius:4px;margin-top:6px}"
89                       "QGroupBox::title{subcontrol-origin:margin;left:8px}")
90     v3 = QVBoxLayout(g3)
91     self._btn(v3, "Iniciar medida", self.start, "#1a6b3a")
92     self._btn(v3, "Parar", self.stop, "#7a2a2a")
93     ll.addWidget(g3)
94
95     g4 = QGroupBox("Análisis")
96     v4 = QVBoxLayout(g4)
97     self._btn(v4, "Depth Profile", self.depth_profile)
98     self._btn(v4, "Ver Comparative", self.show_compare)
99     self._btn(v4, "Mapping 2D", self.map2d)
100    ll.addWidget(g4)
101
102    self.progress = QProgressBar()
103    self.progress.setVisible(False)
104    ll.addWidget(self.progress)
105    ll.addStretch()
106
107    right_col = QWidget()
108    rc = QVBoxLayout(right_col)
109    rc.setContentsMargins(0, 0, 0, 0); rc.setSpacing(4)
110    rl.addWidget(right_col, stretch=1)
111
112    self.tabs = QTabWidget(); self.tabs.setDocumentMode(True)
113    rc.addWidget(self.tabs, stretch=1)
114
115    self.tab_measure = MeasureTab()
116    self.tab_depth = PlotTab()
117    self.tab_compare = CompareTab()
118    self.tab_map = PlotTab()

```

```

119
120     self.tabs.addTab(self.tab_measure, "Medida")
121     self.tabs.addTab(self.tab_depth, "Depth Profile")
122     self.tabs.addTab(self.tab_compare, "Comparative")
123     self.tabs.addTab(self.tab_map, "Mapping 2D")
124
125     # conectar señal del boton Añadir
126     self.tab_compare.add_sim_requested.connect(self._on_add_sim)
127
128     save_bar = QFrame()
129     save_bar.setFrameShape(QFrame.Shape.StyledPanel)
130     save_bar.setStyleSheet("QFrame{background:#1a1a2e;border-top:1px solid
↵ #444466;}")
131     save_bar.setFixedHeight(46)
132     sb = QHBoxLayout(save_bar)
133     sb.setContentsMargins(10, 4, 10, 4); sb.setSpacing(8)
134     sb.addWidget(QLabel("Nombre:"))
135     self.save_name = QLineEdit("medida")
136     self.save_name.setFixedWidth(140)
137     self.save_name.setPlaceholderText("nombre base")
138     sb.addWidget(self.save_name)
139     for label, slot in [("TXT", self.save_txt),
140                        ("Excel", self.save_xlsx),
141                        ("ASCII", self.save_ascii)]:
142         b = QPushButton(label); b.setFixedHeight(30)
143         b.clicked.connect(slot); sb.addWidget(b)
144     sb.addSpacing(16)
145     load_btn = QPushButton("Cargar sesion")
146     load_btn.setFixedHeight(30)
147     load_btn.clicked.connect(self.load_session)
148     sb.addWidget(load_btn)
149     sb.addStretch()

```

```

150         rc.addWidget(save_bar)
151
152         self.setStatusBar(QStatusBar())
153         self.log("Desconectado")
154
155     def _btn(self, layout, text, slot, color=None):
156         btn = QPushButton(text)
157         if color:
158             btn.setStyleSheet(f"QPushButton{{background:{color};color:white;border-radius:4
↪ px}}")
159             f"QPushButton: hover{{background:{color}bb}}")
160         btn.clicked.connect(slot)
161         layout.addWidget(btn)
162         return btn
163
164     def _append_spectrum(self, data, session):
165         n = len(self.saved) + 1
166         name = f"S{n}"
167         entry = {"data": data, "name": name, "session": session}
168         self.saved.append(entry)
169         self._peak_cache.pop(len(self.saved) - 1, None)
170         color = COLORS[(n - 1) % len(COLORS)]
171         self.tab_compare.add_spectrum_row(
172             name, session, color,
173             on_change=self._redraw_compare,
174             on_delete=self._make_delete_fn(entry)
175         )
176
177     def _make_delete_fn(self, entry_ref):
178         """Cierra sobre la referencia al dict, no sobre el índice."""
179         def delete_fn(check=False):
180             for i, sp in enumerate(self.saved):

```

```

181         if sp is entry__ref:
182             self.__delete_spectrum(i)
183             return
184     return delete_fn
185
186 def __delete_spectrum(self, idx):
187     if idx >= len(self.saved): return
188     self.saved.pop(idx)
189     self.__peak_cache.clear()
190
191     __, row, __ = self.tab_compare.checkboxes.pop(idx)
192     row.deleteLater()
193
194     for i, sp in enumerate(self.saved):
195         sp["name"] = f"S{i + 1}"
196         color = COLORS[i % len(COLORS)]
197         self.tab_compare.update_row_name(i, sp["name"], color)
198         cb, __, set_lbl = self.tab_compare.checkboxes[i]
199         cb.setToolTip(f"Conjunto: {sp['session']}")
200         set_lbl.setText(sp["session"][:6])
201
202     # Reconectar botones X con las nuevas referencias
203     for i, sp in enumerate(self.saved):
204         __, row, __ = self.tab_compare.checkboxes[i]
205         del_btn = row.layout().itemAt(2).widget()
206         if del_btn:
207             try: del_btn.clicked.disconnect()
208             except Exception: pass
209             del_btn.clicked.connect(self.__make_delete_fn(sp))
210
211     if self.saved:
212         self.__redraw_compare()

```



```

213         else:
214             self.tab_compare.ax.clear()
215             self.tab_compare._style_ax()
216             self.tab_compare.canvas.draw()
217
218     def __compute_peaks(self, idx):
219         if idx not in self.__peak_cache:
220             s = self.saved[idx]["data"]
221             thresh = 0.08 * (s.max() - s.min())
222             pk, pr = find_peaks(s, prominence=thresh, distance=10)
223             self.__peak_cache[idx] = (pk, pr)
224         return self.__peak_cache[idx]
225
226     def start(self):
227         if not self.specs: self.log("Conecta primero"); return
228         if self.__trig_thread and self.__trig_thread.isRunning():
229             self.log("Medida ya en curso"); return
230
231         self.saved = []
232         self.__peak_cache = {}
233         self.tab_compare.clear_spectra()
234         self.__session_count += 1
235
236         txt = self.spin_shots.text().strip()
237         n_shots = int(txt) if txt.isdigit() else 0
238         edge = self.cmb_edge.currentIndex()
239         accum_mode = self.cmb_accum.currentIndex()
240         normalize = self.chk_normalize.isChecked()
241
242         self.__trig_thread = TriggerThread(
243             specs=self.specs, t_ms=float(self.t.text()),
244             n_shots=n_shots, edge=edge,

```

```

245         normalize=normalize, accum_mode=accum_mode,
246         delay_ref=self.t_re
247     )
248     self.__trig_thread.shot_received.connect(self.__on_shot)
249     self.__trig_thread.finished_all.connect(self.__on_finished)
250     self.__trig_thread.start()
251
252     self.tab_measure.reset()
253     self.tab_measure.set_state("Esperando laser...", "#50fa7b")
254     self.tab_measure.set_target(n_shots)
255     self.tabs.setCurrentWidget(self.tab_measure)
256     shots_txt = str(n_shots) if n_shots > 0 else "inf"
257     modos = ["sin acum.", "suma acum.", "promedio"]
258     self.log(f"Sesion {self.__session_count} - disparos: {shots_txt} modo: {modos[
↪ accum_mode]}")
259
260 def stop(self):
261     if self.__trig_thread:
262         self.__trig_thread.stop(); self.__trig_thread = None
263     self.tab_measure.set_state("Parado", "#ff5555")
264     self.log("Medida parada")
265
266 def __on_shot(self, idx, wl, s):
267     self.wl = wl
268     self.last_spectrum = s
269     base = self.save_name.text().strip() or "medida"
270     base = "".join(c for c in base if c.isalnum() or c in "__-")
271     self.__append_spectrum(s.copy(), f"{base}_{self.__session_count:03d}")
272     self.tab_measure.update_shot(idx, wl, s, overlay=self.chk_overlay.isChecked())
273     self.log(f"Disparo #{idx}")
274
275 def __on_finished(self):

```

```

276     self.tab_measure.set_state("Completado", "#f0a500")
277     self.log("Secuencia completada - guardando...")
278     self.__trig_thread = None
279     self.__autosave()
280
281     def __autosave(self):
282         if not self.saved or self.wl is None: return
283         import os.path
284         base = self.save_name.text().strip() or "medida"
285         base = "".join(c for c in base if c.isalnum() or c in "__-")
286         fname = f"{base}_{self.__session_count:03d}.txt"
287         if os.path.exists(fname):
288             self.log(f"AVISO: '{fname}' ya existe."); return
289         try:
290             with open(fname, "w") as f:
291                 f.write("wl\t" + "\t".join(sp["name"] for sp in self.saved) + "\n")
292                 for j, w in enumerate(self.wl):
293                     vals = "\t".join(f"{sp['data'][j]:.6f}" for sp in self.saved)
294                     f.write(f"{w:.4f}\t{vals}\n")
295                 self.log(f"Auto-guardado: '{fname}' ({len(self.saved)} espectros)")
296         except Exception as e:
297             self.log(f"Error guardando: {e}")
298
299     def __on_add_sim(self):
300         tc = self.tab_compare
301         item = tc.elem_list.currentItem()
302         if item is None:
303             self.log("Selecciona un elemento de la lista primero"); return
304
305         element = item.text()
306
307         Te = tc.sim_Te.value()

```

```

308     Ne    = tc.sim_Ne.value() * 1e17
309     pct   = tc.sim_pct.value()
310     res   = tc.sim_res.value()
311     low_w = tc.sim_low.value()
312     up_w  = tc.sim_up.value()
313     ion   = tc.sim_ion.value()
314     ws    = tc.sim_ws.currentText()
315     wl_ref = self.wl.copy() if self.wl is not None else None
316
317     self.log(f"Simulando {element} ({pct} %) Te={Te:.2f}eV Ne={Ne:.1e} res={res}...")
318     QApplication.processEvents()
319
320     try:
321         from SimulatedLIBS import simulation
322     except ImportError:
323         self.log("SimulatedLIBS no instalado pip install SimulatedLIBS"); return
324
325     try:
326         libs = simulation.SimulatedLIBS(
327             Te=Te, Ne=Ne,
328             elements=[element], percentages=[100],
329             resolution=res, low_w=low_w, upper_w=up_w,
330             max_ion_charge=ion, webscraping=ws,
331         )
332
333         df = libs.get_interpolated_spectrum()
334         wl_col = df.columns[0] # primera columna = longitudes de onda
335         int_col = df.columns[1] # segunda columna = intensidades
336         wl_sim = df[wl_col].to_numpy(dtype=float)
337         s_sim = df[int_col].to_numpy(dtype=float)
338
339         # normalizar a [0, 1]

```

```

340     s_min, s_max = s_sim.min(), s_sim.max()
341     if s_max > s_min:
342         s_sim = (s_sim - s_min) / (s_max - s_min)
343
344     # interpolar al eje del espectrómetro si existe
345     if wl_ref is not None:
346         s_out = np.interp(wl_ref, wl_sim, s_sim)
347     else:
348         s_out = s_sim
349         if self.wl is None:
350             self.wl = wl_sim
351
352     # nombre = "Elemento (porcentaje%)" con parámetros congelados
353     label = f"{element} ({pct} %) Te={Te:.1f} Ne={Ne:.0e}"
354     self._append_spectrum(s_out.copy(), session=label)
355     self.log(f"Simulado: {label}")
356     self._redraw_compare()
357     self.tabs.setCurrentWidget(self.tab_compare)
358
359 except Exception as e:
360     self.log(f"Error simulando {element}: {e}")
361
362 def depth_profile(self):
363     if not self.saved:
364         self.log("No hay espectros haz una medida o carga una sesion"); return
365     if self.wl is None or self.selected_wl is None:
366         self.log("Selecciona lambda_R en Comparative primero"); return
367
368     idx = (np.abs(self.wl - self.selected_wl)).argmin()
369     wl_real = self.wl[idx]
370     visible = self.tab_compare.visible_indices()
371     data = [float(self.saved[i]["data"][idx]) for i in visible]

```

```

372     labels = [self.saved[i]["name"] for i in visible]
373
374     ax = self.tab_depth.ax; ax.clear(); self.tab_depth.__style_ax()
375     ax.plot(range(1, len(data) + 1), data, marker="o", color="#f0a500",
376             linewidth=1.5, markersize=5)
377     ax.set_xticks(range(1, len(data) + 1))
378     ax.set_xticklabels(labels, rotation=45, ha="right", fontsize=7)
379     ax.set_title(f"Depth Profile wl = {wl_real:.2f} nm ({len(data)} espectros)")
380     ax.set_xlabel("Espectro"); ax.set_ylabel("Intensidad")
381     self.tab_depth.canvas.draw()
382     self.tabs.setCurrentWidget(self.tab_depth)
383     self.log(f"Depth Profile en wl={wl_real:.2f} nm")
384
385     def show_compare(self):
386         if not self.saved: self.log("No hay espectros guardados"); return
387         self.__redraw_compare()
388         self.tabs.setCurrentWidget(self.tab_compare)
389
390     def __redraw_compare(self):
391         tc = self.tab_compare; ax = tc.ax
392         ax.clear(); tc.__style_ax()
393
394         visible = tc.visible_indices()
395         for i in visible:
396             sp = self.saved[i]
397             s = sp["data"]
398             color = COLORS[i % len(COLORS)]
399             ax.plot(self.wl, s, label=sp["name"], color=color, linewidth=1.2)
400
401             peaks, props = self.__compute_peaks(i)
402             if len(peaks) > 0:
403                 top = peaks[np.argsort(props["prominences"])[::-1][:6]]

```

```

404         ax.plot(self.wl[top], s[top], "v", color=color,
405                 markersize=6, markeredgecolor="#ffff44", zorder=5)
406     for p in top:
407         ax.annotate(f"{self.wl[p]:.1f}", xy=(self.wl[p], s[p]),
408                 xytext=(0, 8), textcoords="offset points",
409                 ha="center", fontsize=7, color=color)
410
411     mid = len(self.wl) // 2; third = len(self.wl) // 3
412     tc._cursor_r = ax.axvline(tc._wl_r if tc._wl_r is not None else self.wl[mid],
413                             color="#ff5555", linewidth=1.2, linestyle="--", label="lambda_R")
414     tc._cursor_b = ax.axvline(tc._wl_b if tc._wl_b is not None else self.wl[third],
415                             color="#8be9fd", linewidth=1.2, linestyle="--", label="lambda_B
↪ ")
416     ax.legend(facecolor="#1a1a2e", labelcolor="white", fontsize=8)
417     ax.set_title("Comparative")
418     ax.set_xlabel("Longitud de onda (nm)"); ax.set_ylabel("Intensidad norm.")
419
420     for cid in tc._zoom_cids:
421         tc.canvas.mpl_disconnect(cid)
422     tc._zoom_cids = []
423
424     def on_press(event):
425         if event.inaxes != ax or event.xdata is None: return
426         mode = tc._mode
427         if mode == "view": return
428         elif mode == "zoom":
429             tc._zoom_start = (event.xdata, event.ydata)
430             if tc._zoom_patch:
431                 try: tc._zoom_patch.remove()
432                 except Exception: pass
433             tc._zoom_patch = MplRect(
434                 (event.xdata, event.ydata), 0, 0,

```

```

435         linewidth=1, edgecolor="#f0a500", facecolor="#f0a50022")
436     ax.add_patch(tc._zoom_patch)
437 elif mode == "cursor_r":
438     x = self._snap(event.xdata); tc._wl_r = x
439     if tc._cursor_r: tc._cursor_r.set_xdata([x])
440     self._update_selected_wl()
441     self.log(f"lambda_R: {x:.2f} nm")
442     tc.canvas.draw_idle(); tc.update_cursor_display()
443 elif mode == "cursor_b":
444     x = self._snap(event.xdata); tc._wl_b = x
445     if tc._cursor_b: tc._cursor_b.set_xdata([x])
446     self._update_selected_wl()
447     self.log(f"lambda_B: {x:.2f} nm")
448     tc.canvas.draw_idle(); tc.update_cursor_display()
449
450 def on_motion(event):
451     if event.inaxes != ax or event.xdata is None: return
452     mode = tc._mode
453     if mode == "zoom" and tc._zoom_start:
454         x0, y0 = tc._zoom_start
455         if tc._zoom_patch:
456             tc._zoom_patch.set_width(event.xdata - x0)
457             tc._zoom_patch.set_height(event.ydata - y0)
458             tc.canvas.draw_idle()
459     elif mode in ("cursor_r", "cursor_b"):
460         x = self._snap(event.xdata)
461         if mode == "cursor_r" and tc._cursor_r: tc._cursor_r.set_xdata([x])
462         elif mode == "cursor_b" and tc._cursor_b: tc._cursor_b.set_xdata([x])
463         tc.canvas.draw_idle()
464
465 def on_release(event):
466     if tc._mode == "zoom" and tc._zoom_start and event.inaxes == ax:

```



```

467         x0, y0 = tc.__zoom_start
468         x1 = event.xdata or x0; y1 = event.ydata or y0
469         if abs(x1 - x0) > 1 and abs(y1 - y0) > 0.001:
470             ax.set_xlim(min(x0, x1), max(x0, x1))
471             ax.set_ylim(min(y0, y1), max(y0, y1))
472         if tc.__zoom_patch:
473             try: tc.__zoom_patch.remove()
474             except Exception: pass
475             tc.__zoom_patch = None
476         tc.__zoom_start = None
477         tc.canvas.draw_idle()
478
479         tc.__zoom_cids.append(tc.canvas.mpl_connect("button_press_event", on_press))
480         tc.__zoom_cids.append(tc.canvas.mpl_connect("motion_notify_event", on_motion))
481         tc.__zoom_cids.append(tc.canvas.mpl_connect("button_release_event", on_release))
482         tc.canvas.draw(); tc.update_cursor_display()
483
484     def __snap(self, x):
485         return float(self.wl[(np.abs(self.wl - x)).argmin()])
486
487     def __update_selected_wl(self):
488         tc = self.tab_compare
489         if tc.__wl_r is not None and tc.__wl_b is not None:
490             self.selected_wl = tc.__wl_r - tc.__wl_b
491             self.log(f"lambda efectiva (Delta): {self.selected_wl:.2f} nm")
492         elif tc.__wl_r is not None:
493             self.selected_wl = tc.__wl_r
494
495     def map2d(self):
496         if not self.specs: self.log("Conecta primero"); return
497         if self.selected_wl is None:
498             self.log("Selecciona lambda_R en Comparative primero"); return

```

```

499     self.__map_thread = MapThread(
500         specs=self.specs, t_ms=float(self.t.text()),
501         wl=self.wl, selected_wl=self.selected_wl,
502         edge=self.cmb_edge.currentIndex()
503     )
504     self.__map_thread.progress.connect(self.__on_progress)
505     self.__map_thread.done.connect(self.__draw_map)
506     self.__map_thread.start()
507     self.progress.setVisible(True); self.progress.setValue(0)
508     self.tabs.setCurrentWidget(self.tab_map)
509     self.log("Mapping 2D - esperando disparos...")
510
511     def __draw_map(self, cube):
512         self.progress.setVisible(False)
513         ax = self.tab_map.ax; ax.clear(); self.tab_map._style_ax()
514         im = ax.imshow(cube, origin="lower", cmap="plasma")
515         self.tab_map.fig.colorbar(im, ax=ax, label="Intensidad")
516         ax.set_title(f"Mapping 2D  wl = {self.selected_wl:.2f} nm")
517         self.tab_map.canvas.draw()
518         self.log("Mapping 2D completado")
519
520     def __on_progress(self, val):
521         self.progress.setValue(val)
522
523     def load_session(self):
524         paths, __ = QFileDialog.getOpenFileNames(
525             self, "Cargar sesion(es)", "", "Text files (*.txt);;All files (*)")
526         if not paths: return
527         loaded = 0
528         for path in paths:
529             try:
530                 import os.path as op

```

```

531         session_label = op.splitext(op.basename(path))[0]
532         with open(path, "r") as f:
533             header = f.readline().strip().split("\t")
534             data = np.loadtxt(path, delimiter="\t", skiprows=1)
535             if data.ndim == 1 or data.shape[1] < 2:
536                 self.log(f"Formato no reconocido: {path}"); continue
537             wl_loaded = data[:, 0]
538             spectra = [data[:, col] for col in range(1, data.shape[1])]
539             if self.wl is None:
540                 self.wl = wl_loaded
541             elif len(wl_loaded) != len(self.wl):
542                 spectra = [np.interp(self.wl, wl_loaded, s) for s in spectra]
543             for s in spectra:
544                 self.__append_spectrum(s.copy(), session_label)
545                 loaded += 1
546         except Exception as e:
547             self.log(f"Error cargando {path}: {e}")
548         if loaded:
549             self.log(f"Cargados {loaded} espectros")
550             self.__redraw_compare()
551             self.tabs.setCurrentWidget(self.tab_compare)
552
553     def save_txt(self):
554         if not self.saved or self.wl is None: return
555         path, __ = QFileDialog.getSaveFileName(self, "Guardar TXT", "", "Text files (*.txt)")
556         if not path: return
557         with open(path, "w") as f:
558             f.write("wl\t" + "\t".join(sp["name"] for sp in self.saved) + "\n")
559             for j, w in enumerate(self.wl):
560                 vals = "\t".join(f"{sp['data'][j]:.6f}" for sp in self.saved)
561                 f.write(f"{w:.4f}\t{vals}\n")
562         self.log(f"Guardado: {path}")

```

```

563
564 def save__ascii(self):
565     if not self.saved or self.wl is None: return
566     path, __ = QFileDialog.getSaveFileName(self, "Guardar ASCII", "", "ASCII files (*.ascii
↪)")
567     if not path: return
568     with open(path, "w") as f:
569         f.write("# wl " + " ".join(sp["name"] for sp in self.saved) + "\n")
570         for j, w in enumerate(self.wl):
571             vals = " ".join(f"{sp['data'][j]:.6f}" for sp in self.saved)
572             f.write(f"{w:.4f} {vals}\n")
573     self.log(f"Guardado: {path}")
574
575 def save__xlsx(self):
576     if not self.saved or self.wl is None: return
577     try:
578         import openpyxl
579     except ImportError:
580         self.log("pip install openpyxl"); return
581     path, __ = QFileDialog.getSaveFileName(self, "Guardar Excel", "", "Excel files (*.xlsx)
↪")
582     if not path: return
583     if not path.endswith(".xlsx"): path += ".xlsx"
584     wb = openpyxl.Workbook()
585     ws = wb.active; ws.title = "Espectros"
586     ws.cell(row=1, column=1, value="wl (nm)")
587     for i, sp in enumerate(self.saved):
588         ws.cell(row=1, column=i + 2, value=sp["name"])
589     ws.cell(row=2, column=1, value="conjunto")
590     for i, sp in enumerate(self.saved):
591         ws.cell(row=2, column=i + 2, value=sp["session"])
592     for j, w in enumerate(self.wl):

```

```

593         ws.cell(row=j + 3, column=1, value=round(float(w), 4))
594         for i, sp in enumerate(self.saved):
595             ws.cell(row=j + 3, column=i + 2, value=round(float(sp["data"][j]), 6))
596     wb.save(path)
597     self.log(f"Guardado Excel: {path}")

```

Código 1.10. Main.

```

1 if __name__ == "__main__":
2     avaspec1.AVS_Init(-1)
3     app = QApplication(sys.argv)
4     app.setStyle("Fusion")
5     window = App()
6     window.show()
7     sys.exit(app.exec())

```